

Оглавление

1. Введение	2
2. Описание пространственного движения	5
2.1. Дуальные числа	5
2.2. Скользящие векторы	7
2.3. Винты	8
2.3.1. Операции над винтами	9
2.3.2. Дуальный угол между двумя осями	10
2.4. Пространственные перемещения твердого тела с помощью кинематических параметров	11
2.4.1. Дуальные углы	12
2.4.2. Дуальные направляющие косинусы	14
2.4.3. Винт конечного перемещения твердого тела	14
2.5. Формула Родрига и ее дуальный аналог	16
2.6. Практическое применение	17
2.6.1. Пример №1	17
2.6.2. Пример №2	19
2.6.3. Программная реализация на Python	20
3. Кватернионы	22
3.1. Операции над кватернионами	22
3.2. Пространственные перемещения с помощью кватернионов	26
4. Бикватернионы	32
4.1. Операции над бикватернионами	32
4.2. Тригонометрическое представление бикватерниона	36
4.3. Винтовое движение с помощью бикватернионов	40
4.4. Отражение относительно плоскости	43
4.5. Практическое применение	44
4.5.1. Примеры винтового движения	44
4.5.2. Примеры отражения объекта	46
5. Приложения	48
5.1. Приложение №1	48
5.2. Приложение №2	55
5.3. Приложение №3	63
5.4. Приложение №4	75
6. Заключение	82

1. Введение

Для описания вращения твёрдого тела обычно применяют углы Эйлера–Крылова, направляющие косинусы или параметры Родрига–Гамильтона. Долгое время основным способом были углы Эйлера–Крылова. Однако такой способ имеет два недостатка. Во-первых, при некоторых положениях тела кинематические уравнения в углах вырождаются (эффект складывания рамок карданова подвеса), хотя физических ограничений на движение нет. Во-вторых, уравнения содержат тригонометрические функции, что замедляет счёт на бортовых компьютерах.

Развитие систем управления космическими аппаратами, бесплатформенных инерциальных систем и цифровой вычислительной техники потребовало более удобных методов. Наилучшими кинематическими параметрами оказались параметры Родрига–Гамильтона. Их четыре, и они связаны одним уравнением связи (у направляющих косинусов таких уравнений шесть). Главное преимущество так это то, что эти параметры не вырождаются ни при каких положениях тела, а уравнения движения не содержат тригонометрических функций.

Параметры Родрига–Гамильтона удобно записывать с помощью кватернионов. Кватернион — это математический объект из четырёх чисел, введенный Гамильтоном в 1843 г. Оказалось, что операции с кватернионами хорошо описывают конечные повороты твёрдого тела. При этом одни и те же величины выступают и как координаты трёхмерного пространства, и как операторы преобразования, что делает формулы простыми и наглядными.

Модели движения на основе кватернионов по сравнению с классическими обладают тремя преимуществами: в них нет особых точек; нет тригонометрических функций; уравнения имеют симметричную и часто линейную структуру.

На основе кватернионов построены уравнения кинематики вращательного и пространственного движения, уравнения динамики твёрдого тела и гироскопов, уравнения инерциальной навигации. Ещё один важный результат — кватернионная форма регуляризации Кустаанхеймо–Штифеля для задачи движения точки в ньютоновском поле тяготения. Эта регуляризация устраняет особенность уравнений при столкновении и стала основой целого направления в небесной механике.

Для описания общего пространственного движения используют бикватернионы. С их помощью можно решать задачи кинематики роботов-манипуляторов, задачи навигации и пространственной ориентации.

Кватернионные и бикватернионные методы показали свою эффективность в задачах оптимального управления движением. Они позволяют строить аналитические решения там, где классические подходы не работают, и повышают точность и скорость численных расчётов.

Цель работы

Данная работа посвящена применению бикватернионов для моделирования пространственных движений. В основе работы лежат принципы винтового движения, которое представляет собой одновременный поворот и перенос тела в пространстве. Кроме того, в работе представлены примеры отражения геометрического объекта относительно плоскости с использованием бикватернионов. Конечным результатом проделанной работы является реализация ряда программ на языке Julia, которые позволяют свободно использовать операции над бикватернионами для вычислений и выводят наглядную визуализацию с использованием интерактивных пакетов Julia.

Методы исследования

Работа построена на гиперкомплексных числах. Основным математическим инструментом служат кватернионы Гамильтона и бикватернионы. Выбор этих средств обусловлен их эффективностью при описании пространственного движения твёрдого тела и геометрических преобразований.

Кватернионные формулы используются для описания вращений. Применяется алгебра кватернионов в нотации однородных координат, включая операции умножения, сопряжения и скалярного произведения. Вращение точки или вектора задаётся сэндвич-формулой с единичным кватернионом. Вычисление параметров вращения по матрице поворота выполняется по алгоритму, изложенному в работе Лендзела [5]

Для описания общего винтового движения применяются бикватернионы. В основе лежит принцип перенесения Котельникова–Штуди, который заключается в том, что формулы для вращений обобщаются на винтовое движение заменой действительных углов и векторов на дуальные. Геометрические объекты — точка, прямая, плоскость представляются в виде бикватернионов. Их движение под действием винтового преобразования вычисляется через бикватернионные сэндвич-формулы.

Теоретические построения проверяются численно. Для этого создан ряд программ на языке Julia, реализующие бикватернионы с необходимыми операциями и функциями. Расчёты иллюстрируются визуализацией примеров винтового движения и отражения объекта.

Актуальность работы

Задачи описания пространственного движения твёрдого тела возникают в навигации, управлении летательными аппаратами, робототехнике, механике космического полёта и компьютерной графике. Традиционный подход с использованием углов Эйлера–Крылова обладает известными недостатками: вырождением кинематических уравнений при некоторых положениях тела и присутствием тригонометрических функций, замедляющих вычисления на бортовых компьютерах.

Применение кватернионов и бикватернионов позволяет устранить эти недостатки. Кватернионные способы описания движения не имеют особых точек, не содержат тригонометрических функций и обладают симметричной структурой уравнений. Это упрощает как аналитическое, так и численное решение задач механики и управления.

Бикватернионы дают единые формулы для описания винтового движения. Это делает бикватернионный подход удобным средством для решения задач кинематики пространственных механизмов, расчёта движений роботов-манипуляторов и моделирования в компьютерной графике.

Структура работы

Данная выпускная квалификационная работа бакалавра состоит из введения, трех основных глав, заключения, списка литературы и приложений.

Введение содержит обоснование выбора темы. Описывается, насколько она актуальна. Формулируется главная цель работы и показывается, какую ценность она представляет. Также перечисляются публикации, связанные с исследованием, и указывается, где и когда проходили апробации результатов.

В первой главе описываются основные концепции теории винтов и их применимость. Приведены примеры применения формулы Родрига для описания винтового движения прямой в пространстве. Реализован один из примеров с помощью программной реализации на языке Python.

Во второй главе изложены основы кватернионной алгебры: основные операции над кватернионами, формулы винтового движения и их применимость в контексте винтового движения.

В третьей главе с помощью бикватернионной алгебры описываются пространственные перемещения объекта и отражение относительно плоскости. Объясняется принцип перенесения Котельникова-Штуди в контексте перехода от кватернионов к бикватернионам и применимости для винтового движения. Приведен ряд программных реализаций с использованием основных функций бикватернионов, функций и визуализацией на языке Julia.

В заключении приведены выводы по проделанной работе и о дальнейших направлениях исследования.

Апробация работы

...

2. Описание пространственного движения

Винтовое исчисление — это раздел механики, изучающий винты. Винт в свою очередь это объект, хранящий в себе информацию о векторной и моментной частях движения в трехмерном пространстве. Основы теории были заложены в конце XIX века, наиболее подробно теории винтов изложена в трудах Ф.М. Диментберга [12], Ю.Н. Челнокова [8] и В.Н. Гордеева [4].

2.1. Дуальные числа

Дуальные числа — это числа вида:

$$U = u + \varepsilon u^o, \varepsilon^2 = 0,$$

где $u, u^o \in \mathbb{R}$, а ε является символом Клиффорда для которого выполняется определяющее свойство $\varepsilon^2 = 0$ при $\varepsilon \neq 0$. В этой записи u и u^o называются соответственно *главной* и *моментной* частями. Благодаря свойствам символа ε , дуальные числа также называют *параболическими числами Клиффорда*. Важно заметить, что при $u \neq 0$ число представляется через параметр $p = \frac{u^o}{u}$:

$$U = u(1 + \varepsilon p) = ue^{\varepsilon p}.$$

Как и с любыми другими числами, над дуальными числами можно проводить операции, такие как:

1. $U \pm K = (u \pm k) + \varepsilon(u^o \pm k^o)$ — сложение и вычитание,
2. $UK = uk + \varepsilon(uk^o + u^ok)$ — умножение,
3. $\frac{U}{K} = \frac{u}{k} + \varepsilon \frac{u^ok - uk^o}{k^2}$ — деление с учетом $k \neq 0$,
4. $U^n = u^n + \varepsilon n u^o u^{n-1}$, $U^{1/n} = u^{1/n} + \varepsilon \frac{u^o}{n} u^{1/n-1}$ — возведение в степень и извлечение корня.

Также нельзя делить число с ненулевой главной частью на число с нулевой главной частью, потому что это приведет к обнулению выражения, аналогично и для произведения.

В дальнейшем мы будем рассматривать винтовые преобразования, где дуальные числа понадобятся для описания движения твердого тела.

Если $U = u + \varepsilon u^o, \varepsilon^2 = 0$, то разложение любой аналитической функции $F(U)$ имеет следующий вид:

$$F(U) = F(u + \varepsilon u^o) = f(u) + \varepsilon u^o \frac{df(u)}{du},$$

где $f(u) = F(u)$ — главная часть функции.

Также, если функция зависима от дуальных параметров $K_1 = k_1 + \varepsilon k_1^o, \dots, K_n = k_n + \varepsilon k_n^o$, то;

$$F(U, K_1, \dots, K_n) = f(u) + \varepsilon [u^o f'(u) + f^o(u)],$$

где

$$f(u) = F(u, k_1, \dots, k_n), \quad f'(u) = \frac{\partial f}{\partial u},$$

$$f^o(u) = k_1^o \frac{\partial f}{\partial k_1} + \dots + k_n^o \frac{\partial f}{\partial k_n}.$$

Рассмотрим примеры элементарных функций:

$$\sin U = \sin u + \varepsilon u^o \cos u,$$

$$\cos U = \cos u - \varepsilon u^o \sin u,$$

$$\ln U = \ln u + \varepsilon \frac{u^o}{u}, \quad u > 0,$$

$$e^U = e^u (1 + \varepsilon u^o).$$

Для функции дуальной переменной производная будет иметь следующий вид:

$$\frac{dF(U)}{dU} = f'(u) + \varepsilon [u^o f''(u) + f^{o'}(u)].$$

Для интеграла будет верно:

$$\int F(U) dU = \int f(u) du + \varepsilon [u^o f(u) + \int f^o(u) du] + C,$$

где $C = c + \varepsilon c^o$ — дуальная постоянная.

В случае и с интегрированием, и с дифференцированием с дуальной переменной можно свести к обычному:

$$\frac{dF(U)}{dU} = \frac{\partial F(U)}{\partial u}, \quad \int F(U) dU \equiv \int F(U) du.$$

Смысл в том, что все привычные прежде формулы остаются верными и для дуальных переменных. Их функции определяются главной частью $f(u)$, а дуальная часть получается в результате производной от главной части по главному аргументу и если есть, то через параметры.

2.2. Скользящие векторы

В евклидовом пространстве выделяют три вида векторов:

1. Свободный вектор — характеризуется только направлением и длиной.
2. Скользящий вектор — привязан к прямой и может совершать перемещение только вдоль этой прямой.
3. Неподвижный вектор — привязан к конкретной точке приложения.

В контексте скользящего вектора, также упоминаются координаты Плюккера. Скользящий вектор может быть задан шестеркой чисел в трехмерном пространстве:

$$\mathbf{R} = \mathbf{r} + \varepsilon \mathbf{r}^o = \begin{bmatrix} r_1 + \varepsilon r_1^o \\ r_2 + \varepsilon r_2^o \\ r_3 + \varepsilon r_3^o \end{bmatrix},$$

где $\mathbf{r} = (r_1, r_2, r_3)^T$ является главной частью, отвечающей за направление и длину, а $\mathbf{r}^o = (r_1^o, r_2^o, r_3^o)^T$ моментной частью (момент относительно начала координат).

Главная и моментная части связаны, поэтому моментную часть можно выразить через радиус-вектор ρ любой точки на линии вектора:

$$r_1^o = \rho_2 r_3 - r_2 \rho_3, \quad r_2^o = \rho_3 r_1 - r_3 \rho_1, \quad r_3^o = \rho_1 r_2 - r_1 \rho_2,$$

а в векторной форме:

$$\mathbf{r}^o = \boldsymbol{\rho} \times \mathbf{r}.$$

Существует условие Плюккера, которое заключается в том, что для одного скользящего вектора будет выполняться условие ортогональности:

$$r_1 r_1^o + r_2 r_2^o + r_3 r_3^o = 0 \quad \Leftrightarrow \quad \mathbf{r} \cdot \mathbf{r}^o = 0.$$

По координатам Плюккера также можно найти точку на линии вектора, которая будет являться ближайшей к началу координат:

$$d_1 = \frac{r_2 r_3^o - r_2^o r_3}{r_1^2 + r_2^2 + r_3^2}, \quad d_2 = \frac{r_3 r_1^o - r_3^o r_1}{r_1^2 + r_2^2 + r_3^2}, \quad d_3 = \frac{r_1 r_2^o - r_1^o r_2}{r_1^2 + r_2^2 + r_3^2}.$$

В векторной форме:

$$\mathbf{d} = \frac{\mathbf{r} \times \mathbf{r}^o}{\|\mathbf{r}\|^2}.$$

У найденной точки будет ряд свойств:

- Радиус вектор $\mathbf{d}$ перпендикулярен главному вектору $\mathbf{r}$.
- Векторы $\mathbf{r}$, $\mathbf{r}^o$ и $\mathbf{d}$ попарно ортогональны.

Сложение плюккеровых координат является их преимуществом так, как выполняется покомпонентно:

$$\mathbf{R}_\Sigma = \mathbf{r}_\Sigma + \varepsilon \mathbf{r}_\Sigma^o,$$

где при сложении главный вектор $\mathbf{r}_\Sigma = \sum \mathbf{r}_k$, а главный момент $\mathbf{r}_\Sigma^o = \sum \mathbf{r}_k^o$.

Отметим, что после сложения условие Плюккера может нарушиться:

$$\mathbf{r}_\Sigma \cdot \mathbf{r}_\Sigma^o \neq 0.$$

Введем также такие понятия, как инвариант дуального вектора:

$$I = \mathbf{r}_\Sigma \cdot \mathbf{r}_\Sigma^o = r_1 r_1^o + r_2 r_2^o + r_3 r_3^o.$$

и параметр дуального вектора:

$$[\mathbf{R}] = \frac{\mathbf{r}_\Sigma \cdot \mathbf{r}_\Sigma^o}{\|\mathbf{r}_\Sigma\|^2}.$$

2.3. Винты

Винт, а в терминологии Котельникова упоминается как мотор, является математическим объектом, который может иметь представление в виде дуального вектора:

$$\mathbf{R} = \mathbf{r} + \varepsilon \mathbf{r}^o = \begin{bmatrix} r_1 + \varepsilon r_1^o \\ r_2 + \varepsilon r_2^o \\ r_3 + \varepsilon r_3^o \end{bmatrix},$$

где $\varepsilon^2 = 0$.

К частным случаям относят скользящий вектор, винт и пару сил. Отметим, что в случае с скользящим вектором момент ортогонален вектору, у винта момент будет коллинеарен вектору, а у пары сил есть чистый момент, который не связан к вектору силы.

В теории винтов вводятся такие понятия, как норма винта или как ее еще называют дуальная величина:

$$\|\mathbf{R}\| = (\mathbf{r} + \varepsilon \mathbf{r}^o) \cdot (\mathbf{r} + \varepsilon \mathbf{r}^o) = \|\mathbf{r}\|^2 + 2\varepsilon(\mathbf{r} \cdot \mathbf{r}^o).$$

Модуль винта(дуальное число):

$$|\mathbf{R}| = \sqrt{\|\mathbf{R}\|} = \|\mathbf{r}\| \left(1 + \varepsilon \frac{\mathbf{r} \cdot \mathbf{r}^o}{\|\mathbf{r}\|^2} \right) = \|\mathbf{r}\|(1 + \varepsilon[\mathbf{R}]),$$

где $[\mathbf{R}] = \frac{\mathbf{r} \cdot \mathbf{r}^o}{\|\mathbf{r}\|^2}$ является параметром винта.Заметим, что понятие модуля определяется только при $\mathbf{r} \neq \mathbf{0}$.

Единичный винт представляет собой ось с направлением в пространстве и обозначается, как $\mathbf{E}$. Для него будет верна следующая запись:

$$\mathbf{E} = \mathbf{e} + \varepsilon \mathbf{e}^o,$$

где $e_1^2 + e_2^2 + e_3^2 = 1$, $e_1 e_1^o + e_2 e_2^o + e_3 e_3^o = 0$.Такой винт обладает единичной векторной частью $\|\mathbf{e}\| = 1$ и параметром равным нулю $\mathbf{e} \cdot \mathbf{e}^o = 0$.

Винт имеет два основных представления:

1. Через плюккеровы координаты или бивекторное:

$$\mathbf{R} = \mathbf{r} + \varepsilon \mathbf{r}^o.$$

2. Через ось и дуальный модуль (винтовое представление):

$$\mathbf{R} = R\mathbf{E},$$

где $R = |\mathbf{R}| = \|\mathbf{r}\|(1 + \varepsilon[\mathbf{R}])$.

2.3.1. Операции над винтами

Сложение винтов выполняется покомпонентно:

$$\mathbf{R} = \mathbf{R}_1 + \mathbf{R}_2 + \dots + \mathbf{R}_n,$$

при условиях $\mathbf{r} = \mathbf{r}_1 + \mathbf{r}_2 + \dots + \mathbf{r}_n$ и $\mathbf{r}^o = \mathbf{r}_1^o + \mathbf{r}_2^o + \dots + \mathbf{r}_n^o$.

Сложение винтов является коммутативным и ассоциативным. Скалярное произведение дистрибутивно:

$$\mathbf{R} \cdot \mathbf{S} = (\mathbf{R}_1 + \mathbf{R}_2 + \dots) \cdot \mathbf{S} = \mathbf{R}_1 \cdot \mathbf{S} + \mathbf{R}_2 \cdot \mathbf{S} + \dots$$

Умножение винта может быть произведено на разные виды чисел. Например, для умножения на действительное число будет верно:

$$a\mathbf{R} = a\mathbf{r} + \varepsilon a\mathbf{r}^o,$$

здесь сохраняется ось винта и при $a > 0$ направление сохраняется, при $a < 0$ меняется на противоположное.

Для умножения на дуальное число вида $A = a + \varepsilon a^o$:

$$A\mathbf{R} = (a + \varepsilon a^o)(\mathbf{r} + \varepsilon \mathbf{r}^o) = a\mathbf{r} + \varepsilon(a\mathbf{r}^o + a^o\mathbf{r}),$$

а в винтовом представлении примет следующий вид:

$$A\mathbf{R} = (a + \varepsilon a^o)\|\mathbf{r}\|e^{\varepsilon[\mathbf{R}]} \mathbf{E} = a\|\mathbf{r}\|e^{\varepsilon([\mathbf{R}] + a^o/a)} \mathbf{E}.$$

Перейдем к скалярному произведению винтов. Для двух винтов вида $\mathbf{R}_1 = \mathbf{r}_1 + \varepsilon \mathbf{r}_1^o$ и $\mathbf{R}_2 = \mathbf{r}_2 + \varepsilon \mathbf{r}_2^o$ будет верна следующая запись:

$$\mathbf{R}_1 \cdot \mathbf{R}_2 = \mathbf{r}_1 \cdot \mathbf{r}_2 + \varepsilon(\mathbf{r}_1 \cdot \mathbf{r}_2^o + \mathbf{r}_1^o \cdot \mathbf{r}_2).$$

В винтовом представлении:

$$\mathbf{R}_1 \cdot \mathbf{R}_2 = R_1 R_2 \cos \Gamma,$$

для $\mathbf{R}_1 = R_1 \mathbf{E}_1$, $\mathbf{R}_2 = R_2 \mathbf{E}_2$ и дуальный угол между осями $\Gamma = \gamma + \varepsilon \gamma^o$. Винтовое произведение также является коммутативным $\mathbf{R}_1 \cdot \mathbf{R}_2 = \mathbf{R}_2 \cdot \mathbf{R}_1$.

Для винтового же произведения будет верно:

$$\mathbf{R}_1 \times \mathbf{R}_2 = \mathbf{r}_1 \times \mathbf{r}_2 + \varepsilon(\mathbf{r}_1 \times \mathbf{r}_2^o + \mathbf{r}_1^o \times \mathbf{r}_2),$$

а винтовое представление примет следующий вид:

$$\mathbf{R}_1 \times \mathbf{R}_2 = R_1 R_2 \sin \Gamma \mathbf{E}_{12},$$

где $\mathbf{E}_{12}$ является единичным винтом, направленным по общему перпендикуляру. Здесь нет коммутативности, как в случае со скалярным произведением, винтовое произведение обладает свойством антикоммутативности $\mathbf{R}_1 \times \mathbf{R}_2 = -\mathbf{R}_2 \times \mathbf{R}_1$. Также ось результирующего винта (произведения) перпендикулярна осям исходных и модуль равен произведению дуальных модулей на синус дуального угла.

К операциям над винтами также относят проекцию винта на ось. Для проекции винта $\mathbf{R}$ на ось, заданную единичным винтом $\mathbf{E}$ будет выполняться:

$$\mathbf{R} \cdot \mathbf{E} = r \cos \gamma + \varepsilon r(p \cos \gamma - \gamma^o \sin \gamma)$$

2.3.2. Дуальный угол между двумя осями

Рассмотрим две направленные оси O_1 и O_2 , тогда дуальный угол между ними:

$$\Gamma = \gamma + \varepsilon \gamma^o,$$

где γ является обычным углом между осями ($0 \leq \gamma \leq \pi$), а γ^o кратчайшее расстояние между осями (знак определяется по правилу правой тройки).

Для двух единичных винтов $\mathbf{O}_1 = \mathbf{o}_1 + \varepsilon \mathbf{o}_1^o$ и $\mathbf{O}_2 = \mathbf{o}_2 + \varepsilon \mathbf{o}_2^o$ будет верно:

$$\cos \Gamma = \mathbf{O}_1 \cdot \mathbf{O}_2 = \mathbf{o}_1 \cdot \mathbf{o}_2 + \varepsilon(\mathbf{o}_1 \cdot \mathbf{o}_2^o + \mathbf{o}_1^o \cdot \mathbf{o}_2),$$

а координатное выражение примет вид:

$$\cos \gamma = o_{11}o_{21} + o_{12}o_{22} + o_{13}o_{23},$$

$$\gamma^o \sin \gamma = o_{11}o_{21}^o + o_{12}o_{22}^o + o_{13}o_{23}^o + o_{11}^o o_{21} + o_{12}^o o_{22} + o_{13}^o o_{23}.$$

Дуальный угол также обладает рядом тригонометрических свойств:

- $\sin \Gamma = \sin \gamma + \varepsilon \gamma^o \cos \gamma,$
- $\cos \Gamma = \cos \gamma - \varepsilon \gamma^o \sin \gamma,$
- $\operatorname{tg} \Gamma = \operatorname{tg} \gamma + \frac{\varepsilon \gamma^o}{\cos^2 \gamma}.$

Рассмотрев основные тригонометрические свойства можно перейти к координатному представлению винтов. Для описания такого представления используются дуальные координаты. Винт $\mathbf{R}$ можно представить через три дуальные координаты, а если точнее проекции на оси прямоугольной системы координат. Соответственно, для X ;

$$X = x + \varepsilon x^o = R \cos \Gamma_X = r[\cos \gamma_X + \varepsilon(p \cos \gamma_X - \gamma_X^o \sin \gamma_X)].$$

Для Y :

$$Y = y + \varepsilon y^o = R \cos \Gamma_Y = r[\cos \gamma_Y + \varepsilon(p \cos \gamma_Y - \gamma_Y^o \sin \gamma_Y)].$$

Для Z :

$$Z = z + \varepsilon z^o = R \cos \Gamma_Z = r[\cos \gamma_Z + \varepsilon(p \cos \gamma_Z - \gamma_Z^o \sin \gamma_Z)].$$

Здесь $\Gamma_X, \Gamma_Y, \Gamma_Z$ являются дуальными углами между осью винта и координатными осями.

Винт также возможно представить в виде винта:

$$\mathbf{R} = X\mathbf{i} + Y\mathbf{j} + Z\mathbf{k} = (x + \varepsilon x^o)\mathbf{i} + (y + \varepsilon y^o)\mathbf{j} + (z + \varepsilon z^o)\mathbf{k}.$$

К основным свойствам подобного представления относят:

- Квадрат модуля:

$$R^2 = X^2 + Y^2 + Z^2 = (x^2 + y^2 + z^2) + 2\varepsilon(xx^o + yy^o + zz^o).$$

- Основное тождество для дуальных направляющих косинусов:

$$\cos^2 \Gamma_X + \cos^2 \Gamma_Y + \cos^2 \Gamma_Z = 1,$$

впоследствии это тождество распадается на два вещественных уравнения $\cos^2 \gamma_X + \cos^2 \gamma_Y + \cos^2 \gamma_Z = 1$ и $\gamma_X^o \cos \gamma_X \sin \gamma_X + \gamma_Y^o \cos \gamma_Y \sin \gamma_Y + \gamma_Z^o \cos \gamma_Z \sin \gamma_Z = 0$.

Для скалярного произведения в координатах будет верно:

$$\mathbf{R}_1 \cdot \mathbf{R}_2 = X_1 X_2 + Y_1 Y_2 + Z_1 Z_2 = (x_1 x_2 + y_1 y_2 + z_1 z_2) + \varepsilon(x_1 x_2^o + x_1^o x_2 + y_1 y_2^o + y_1^o y_2 + z_1 z_2^o + z_1^o z_2),$$

а для винтового:

$$\mathbf{R}_1 \times \mathbf{R}_2 = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ X_1 & Y_1 & Z_1 \\ X_2 & Y_2 & Z_2 \end{vmatrix}.$$

2.4. Пространственные перемещения твердого тела с помощью кинематических параметров

Мы будем рассматривать движение свободного твердого тела относительно произвольной неподвижной системы координат $OX_1 X_2 X_3$. Основное отличие от тела с закрепленной точкой в том, что здесь тело может как вращаться, так и перемещаться поступательно.

Согласно теореме Шаля [8]:

«Всякое перемещение свободного твердого тела из одного положения в другое может быть получено посредством поступательного перемещения тела вместе с произвольно выбранным полюсом и поворота тела вокруг некоторой оси, проходящей через этот полюс.»

Следовательно поступательная часть зависит от выбора полюса, так как разные полюса дают разные векторы переноса, а вращательная часть в свою очередь не зависит от выбора.

Для бесконечно малых перемещений теорема Шаля подводит к понятию винтового движения. Винтовое движение — это комбинация вращения вокруг некоторой оси с угловой скоростью ω и поступательного движения вдоль той же оси со скоростью v . Параметр винта(шаг) имеет следующий вид:

$$h = \frac{v}{\omega},$$

при $\omega \neq 0$. Если же $\omega = 0$, то движение будет чисто поступательным, если $v = 0$ то чисто вращательное.

При движении твердого тела мгновенная винтовая ось(ось, вокруг которой происходит вращение и вдоль которой поступательное движение) меняет свое положение в пространстве. Неподвижный аксоид — это место мгновенных винтовых осей в неподвижной системе координат, а подвижный — место мгновенных винтовых осей в системе координат, связанной с телом.

Оба аксоида представляют собой линейчатые поверхности, которые не разветвляются, т.е. образованы движением прямой линии. В частных случаях аксоиды могут вырождаться в конические поверхности при движении с закрепленной точкой и цилиндрические поверхности при плоском движении.

К традиционным конематическим параметрам, которые используют для описания движения твердого тела, относят:

- Поступательные параметры — декартовы координаты выбранного полюса, всего 3 параметра.
- Вращательные параметры — углы, задающие ориентацию тела, также всего 3 параметра.

2.4.1. Дуальные углы

Для задания ориентации твердого тела используются углы Эйлера ψ, θ, φ и углы Крылова, их еще называют самолетными.

Для описания свободного твердого тела, которое может и вращаться, и поступательно перемещаться, логичнее всего использовать дуальные аналоги обычных углов — дуальные углы Эйлера-Крылова. Дуальный угол же это объект вида:

$$\Theta = \theta + \varepsilon \theta^o,$$

где θ является обычным углом поворота, θ^o линейным перемещением вдоль оси вращения, ε дуальной единицей.

Если же рассматривать отдельно дуальные углы Эйлера, то здесь важно отметить, что происходит переход из начального положения в произвольное конечное с помощью последовательности трех винтовых перемещений, которые как раз и задаются этими углами.

- Дуальный угол прецессии:

$$\Psi = \psi + \varepsilon\psi^o,$$

где вращение происходит вокруг оси X_3 или Y_3 .

- Дуальный угол нутации:

$$\Theta = \theta + \varepsilon\theta^o,$$

где вращение происходит вокруг новой оси Y_1' .

- Дуальный угол собственного вращения:

$$\Phi = \varphi + \varepsilon\varphi^o,$$

где вращение происходит вокруг конечной оси Y_3 .

Важно не забывать о правилах знаков при рассмотрении подобных пространственных перемещений. Углы ψ, θ, φ положительны при вращении против часовой стрелки, если смотреть относительно конца соответствующей оси. Линейные перемещения $\psi^o, \theta^o, \varphi^o$ положительны в направлении соответствующей оси.

Говоря о недостатках таких углов можно заметить аналогию с обычными углами Эйлера, здесь присутствует отсутствие симметрии. При малых отклонениях от начального положения малыми будут только θ и сумма $\psi + \varphi$, также отдельные углы ψ и φ могут принимать произвольные значения. Это все приводит к сингулярностям, например, при $\theta = 0$, когда оси X_3 и Y_3 совпадают, собственное вращение становится неразличимым.

Как и ранее упоминалось, существуют самолетные углы — дуальные угла Крылова. Они являются альтернативой, учитывая все недостатки углов Эйлера. Они называются самолетными так как получили применение в авиации, поэтому будем использоваться соответствующие наименования:

- Дуальный угол рыскания:

$$\Psi = \psi + \varepsilon\psi^o,$$

где вращение происходит вокруг нормальной оси X_2 .

- Дуальный угол тангажа:

$$\Theta = \theta + \varepsilon\theta^o,$$

где вращение происходит вокруг поперечной оси.

- Дуальный угол крена:

$$\Phi = \varphi + \varepsilon\varphi^o,$$

где вращение происходит вокруг продольной оси.

В случае с углами Крылова отсутствует проблема сингулярностей и при малых отклонениях от начального положения малыми будут все три дуальных угла.

2.4.2. Дуальные направляющие косинусы

Для описания ориентации твердого тела вместо углов можно использовать направляющие косинусы. В дуальном варианте это позволяет одновременно описывать и ориентацию, и положение. Рассмотрим две системы координат, X — неподвижная система $OX_1X_2X_3$ и Y — система, связанная с телом $AY_1Y_2Y_3$. Дуальный направляющий косинус — это скалярное произведение единичных векторов осей:

$$C_{ik} = \mathbf{Y}_i \cdot \mathbf{X}_k = \cos \Gamma_{ik},$$

где дуальный угол между осями Y_i и X_k является $\Gamma_{ik} = \gamma_{ik} + \varepsilon \gamma_{ik}^o$.

Если разложить дуальный косинус:

$$C_{ik} = \underbrace{\cos \gamma_{ik}}_{c_{ik}} + \varepsilon \underbrace{(-\gamma_{ik}^o \sin \gamma_{ik})}_{c_{ik}^o},$$

то можно увидеть, что γ_{ik} обычный угол между осями и γ_{ik}^o кратчайшее расстояние между осями.

Матрица для дуальных направляющих косинусов имеет следующий вид:

$$\mathbf{C} = \|C_{ik}\| = \|c_{ik} + \varepsilon c_{ik}^o\| = \begin{pmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{pmatrix}$$

К тому же, у Лурье [13] уже были выведены формулы для элементов матрицы поворота через параметры Родрига-Гамильтона:

$$a_{sk} = \delta_{sk} \left(1 - 2 \sum_{q=1}^3 \lambda_q^2 \right) + 2\lambda_0 \sum_{t=1}^3 \varepsilon_{tsk} \lambda_t + 2\lambda_s \lambda_k.$$

2.4.3. Винт конечного перемещения твердого тела

Согласно теореме Шаля, любое перемещение сводобного твердого тела можно представить как винтовое перемещение, т.е. вращение вокруг некоторой оси и поступательное движение вдоль той же оси. Конечное винтовое перемещение описывается дуальным вектором:

$$\Theta = 2\mathbf{E} \operatorname{tg} \frac{\Phi}{2},$$

где $\mathbf{E} = \mathbf{e} + \varepsilon \mathbf{e}^o$ является единичным винтом оси винтового перемещения, $\Phi = \varphi + \varepsilon \varphi^o$ дуальным углом поворота, φ углом поворота вокруг оси, а φ^o поступательным перемещением вдоль оси.

Проекция же винта Θ на оси неподвижной системы X имеет следующий вид:

$$\Theta_i = 2 \operatorname{tg} \frac{\Phi}{2} \cos \Gamma_i, \quad i = 1, 2, 3,$$

где $\Gamma_i = \gamma_i + \varepsilon \gamma_i^o$ — дуальный угол между осью винта и осью X_i .

При рассмотрении конечных перемещений в пространстве также используются дуальные параметры Эйлера-Родрига-Гамильтона и Кейли-Клейна. У Лурье [13], например, параметры вводятся через вектор конечного поворота θ , что демонстрирует связь между параметрами Родрига-Гамильтона и компонентами вектора конечного поворота:

$$\frac{1}{2}\theta_s = \frac{\lambda_s}{\lambda_0}, \quad s = 1, 2, 3.$$

Дуальные параметры Эйлера, которые являются обобщением классических параметров Родрига-Гамильтона, определяются через проекции винта перемещения:

$$\Lambda_0 = \cos \frac{\Phi}{2}, \quad \Lambda_i = \frac{1}{2}\Lambda_0\Theta_i = \sin \frac{\Phi}{2} \cos \Gamma_i, \quad i = 1, 2, 3$$

У дуальных параметров Эйлера-Родрига-Гамильтона каждый параметр является дуальным числом:

$$\Lambda_j = \lambda_j + \varepsilon\lambda_j^o, \quad j = 0, 1, 2, 3,$$

где λ_j являются вещественными параметрами Эйлера, а λ_j^o их дуальными компонентами.

Для дуальных параметров выполняется одно основное тождество:

$$\Lambda_0^2 + \Lambda_1^2 + \Lambda_2^2 + \Lambda_3^2 = 1.$$

Оно же распадается на два вещественных уравнения:

$$\lambda_0^2 + \lambda_1^2 + \lambda_2^2 + \lambda_3^2 = 1,$$

которое является условием нормировки главных частей и условие ортогональности:

$$\lambda_0\lambda_0^o + \lambda_1\lambda_1^o + \lambda_2\lambda_2^o + \lambda_3\lambda_3^o = 0.$$

Сами параметры Кейли-Клейна вводятся через параметры Эйлера:

$$A = \Lambda_0 + i\Lambda_3 = \alpha + \varepsilon\alpha^o,$$

$$B = -\Lambda_2 + i\Lambda_1 = \beta + \varepsilon\beta^o,$$

$$\Gamma = \Lambda_2 + i\Lambda_1 = \gamma + \varepsilon\gamma^o,$$

$$\Delta = \Lambda_0 - i\Lambda_3 = \delta + \varepsilon\delta^o,$$

где $\alpha, \beta, \gamma, \delta$ вещественные параметры Кейли-Клейна, $\alpha^o, \beta^o, \gamma^o, \delta^o$ их дуальные компоненты, а i мнимая единица. Основным их тождеством является $A\Delta - B\Gamma = 1$

2.5. Формула Родрига и ее дуальный аналог

Классическая формула Родрига используется для поворота точки вокруг оси, но также может быть обобщена на случай винтового движения. Сама формула Родрига в компактной векторной форме принимает следующий вид:

$$\mathbf{r}' = \mathbf{r} + \left(1 + \frac{\theta^2}{4}\right)^{-1} \left[\boldsymbol{\theta} \times \left(\mathbf{r} + \frac{1}{2} \boldsymbol{\theta} \times \mathbf{r} \right) \right].$$

Но существуют и альтернативные формы записи, например, через тригонометрические функции:

$$\mathbf{r}' = \mathbf{r} + \sin \varphi (\mathbf{e} \times \mathbf{r}) + (1 - \cos \varphi) [\mathbf{e} \times (\mathbf{e} \times \mathbf{r})].$$

Или более развернутая:

$$\mathbf{r}' = \cos \varphi \mathbf{r} + \sin \varphi (\mathbf{e} \times \mathbf{r}) + (1 - \cos \varphi) (\mathbf{e} \cdot \mathbf{r}) \mathbf{e}.$$

Существует теорема о двух полуоборотах согласно которой:

«Поворот твердого тела на угол φ вокруг некоторой оси эквивалентен двум последовательным полуоборотам тела вокруг осей, пересекающих под прямым углом в одной точке данную ось и образующих между собой угол $\varphi/2$ ».

Для свободного твердого тела (нет закрепленных точек) обобщением формулы Родрига является дуальный аналог, описывающий винтовое движение, как комбинацию вращательного и поступательного действий. Введем дуальный аналог величин, использованных в классической формуле:

- Угол поворота φ заменяется на дуальный угол $\Phi = \varphi + \varepsilon \varphi^\circ$.
- Единичный вектор оси $\mathbf{e}$ на единичный винт оси $\mathbf{E} = \mathbf{e} + \varepsilon \mathbf{e}^\circ$.
- Вектор конечного поворота $\boldsymbol{\theta} = 2 \operatorname{tg}(\varphi/2) \mathbf{e}$ на дуальный вектор конечного перемещения $\boldsymbol{\Theta} = 2 \mathbf{E} \operatorname{tg}(\Phi/2)$.
- Радиус-вектор точки $\mathbf{r}$ на винт точки $\mathbf{R} = \mathbf{r} + \varepsilon \mathbf{r}^\circ$.

В таком случае дуальная формула Родрига будет иметь следующий вид:

$$\mathbf{R}' = \mathbf{R} + \left(1 + \frac{\boldsymbol{\Theta}^2}{4}\right)^{-1} \left[\boldsymbol{\Theta} \times \left(\mathbf{R} + \frac{1}{2} \boldsymbol{\Theta} \times \mathbf{R} \right) \right],$$

или в более развернутой тригонометрической форме:

$$\mathbf{R}' = \cos \Phi \mathbf{R} + \sin \Phi (\mathbf{E} \times \mathbf{R}) + (1 - \cos \Phi) (\mathbf{E} \cdot \mathbf{R}) \mathbf{E}.$$

Существуют частные случаи перемещений в пространстве с использованием этой формулы.

- Чистое вращение ($\varphi^\circ = 0, \mathbf{e}^\circ = 0$):

$$\mathbf{R}' = \mathbf{r}' + \varepsilon \mathbf{r}'^\circ.$$

- Чистая трансляция ($\varphi = 0, \mathbf{e}^\circ = 0$) и при $\varphi = 0, \cos \Phi = 1, \sin \Phi = \varepsilon \varphi^\circ$:

$$\mathbf{R}' = \mathbf{R} + \varepsilon \varphi^\circ (\mathbf{e} \times \mathbf{R}),$$

если с учетом главной и моментной частей:

$$\mathbf{r}' = \mathbf{r}, \quad \mathbf{r}'^\circ = \mathbf{r}^\circ + \varphi^\circ (\mathbf{e} \times \mathbf{r}),$$

что полностью соответствует переносу точки на вектор $\varphi^\circ \mathbf{e}$, т.е. моментная часть изменяется.

- Полуоборот ($\varphi = \pi, \varphi^\circ = 0$):

$$\mathbf{r}' = 2(\mathbf{e} \cdot \mathbf{r})\mathbf{e} - \mathbf{r},$$

$$\mathbf{r}'^\circ = 2(\mathbf{e} \cdot \mathbf{r}^\circ + \mathbf{e}^\circ \cdot \mathbf{r})\mathbf{e} + 2(\mathbf{e} \cdot \mathbf{r})\mathbf{e}^\circ - \mathbf{r}^\circ.$$

Ранее упоминалась теорема о двух полуоборотах, соответственно, у нее тоже есть дуальный аналог:

«Винтовое перемещение тела на дуальный угол $\Phi = \varphi + \varepsilon \varphi^\circ$ относительно оси, единичный винт которой $\mathbf{E}$, эквивалентно двум последовательным полуоборотам, совершаемым относительно прямых с единичными винтами $\mathbf{E}_1$ и $\mathbf{E}_2$, пересекающихся под прямым углом ось $\mathbf{E}$ и образующих между собой дуальный угол $\Phi/2$.

То, как мы применили дуальные аналоги к классической формуле Родрига называется принципом перенесения Котельникова-Штуди.

2.6. Практическое применение

Для описания винтового движения будем следовать принципу Котельникова-Штуди (описанного в разделе 2.5) согласно которому все формулы для конечных поворотов будут иметь смысл и для винтового движения, если в них произвести замену векторов на винты, вещественных чисел на дуальные, углы на дуальные углы и кватернионы на бикватернионы.

2.6.1. Пример №1

Рассмотрим наглядный пример винтового движения [2]. Есть некоторая прямая, которая лежит в плоскости Oxy и проходит через точку. Точка O — это точка начала отсчета. Она будет проходить под углом 90° по отношению к осям Ox и Oy . Направляющий вектор для такой прямой будет следующим:

$$\mathbf{v} = (-1, -1, -2)^T.$$

Также, как выше было упомянуто у нас есть точка O . Она задана следующим образом:

$$O = (0, 0, 0)^T.$$

Теперь вычислим момент. Мы также учитываем, что в ходе вычисления данного примера $\mathbf{v}$ отложен от O , из этого следует, что вычисление момента будет проходить по следующей формуле:

$$\mathbf{m} = \mathbf{p} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 0 & 0 & 0 \\ -1 & -1 & -2 \end{vmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Таким образом соответствующий прямой e винт e' можно вычислить по следующей формуле:

$$\mathbf{e} = \mathbf{v} + \varepsilon \mathbf{m} = (-1, -1, -2)^T + \varepsilon(0, 0, 0)^T = -1i - 1j - 2k.$$

Теперь рассмотрим дуальный угол для поворота прямой. Дуальный угол как выше было упомянуто представляется в виде $A = \alpha + \varepsilon \alpha^\circ$. На данном этапе нужно задать параметры α и α° . Где $\alpha = \pi/2$ будет углом на который мы поворачиваем, а $\alpha^\circ = 1$ будет обозначать величину подъема вдоль оси. Чтобы это реализовать, мы можем использовать формулу Родрига (вывод формулы привеен в разделе 2.5):

$$\mathbf{e}' = \cos A \mathbf{e} + \sin A (\mathbf{E} \times \mathbf{e}) + (1 - \cos A)(\mathbf{E} \cdot \mathbf{e})\mathbf{E},$$

где $\mathbf{E} = [0, 0, 1]$ обозначает единичный винт оси винтового перемещения. В данном случае вокруг оси Oz . Подставим в формулу все полученные и ранее имеющиеся значения:

$$\begin{aligned} \mathbf{e}' &= \cos A (-1, -1, -2) + \sin A ((0, 0, 1) \times (-1, -1, -2)) + (1 - \cos A)((0, 0, 1)(-1, -1, -2))(0, 0, 1) \\ &= (0, 0, -2 + 2 \cos A) + (-\cos A, -\cos A, -2 \cos A) + (\sin A, \sin A, 0) \\ &= \begin{bmatrix} -\cos A + \sin A \\ -\cos A - \sin A \\ -2 \end{bmatrix}. \end{aligned}$$

Получив результат вычислений, мы можем воспользоваться следующими двумя формулами для упрощения вида.

Для синуса:

$$\sin A = \sin(\alpha + \varepsilon \alpha^\circ) = \sin \alpha + \alpha^\circ \cos \alpha \varepsilon.$$

Для косинуса:

$$\cos A = \cos(\alpha + \varepsilon \alpha^\circ) = \cos \alpha - \alpha^\circ \sin \alpha \varepsilon.$$

Используем их для разложения элементов получившегося вектор-столбца и подставим дуальный угол:

$$\begin{aligned} -\cos A + \sin A &= -\cos \pi/2 + \sin \pi/2\varepsilon + \sin \pi/2 + \cos \pi/2\varepsilon = \varepsilon + 1, \\ -\cos A - \sin A &= -\cos \pi/2 + \sin \pi/2\varepsilon - \sin \pi/2 - \cos \pi/2\varepsilon = \varepsilon - 1. \end{aligned}$$

$$\mathbf{e}' = \begin{bmatrix} \varepsilon + 1 \\ \varepsilon - 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ -2 \end{bmatrix} + \varepsilon \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

Исходя из этого, мы получаем следующее:

$$\mathbf{m}' = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \quad \mathbf{v}' = \begin{bmatrix} 1 \\ -1 \\ -2 \end{bmatrix}.$$

Теперь нормализуем изначальный вектор $\mathbf{v}$:

$$\mathbf{v} = [-1/\sqrt{6}, -1/\sqrt{6}, -2/\sqrt{6}].$$

Вновь найдем $\mathbf{e}'$ с учетом нормализации:

$$\mathbf{e}' = \begin{bmatrix} -1/\sqrt{6} \\ -1/\sqrt{6} \\ -2/\sqrt{6} \end{bmatrix} + \varepsilon \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix}.$$

В полученном винте главной частью является направляющий вектор оси. Для однозначной визуализации данной оси в пространстве необходимо вычислить точку, через которую она проходит. Это можно сделать следующим образом:

$$\mathbf{p}'_0 = \frac{\mathbf{v}' \times \mathbf{m}'}{\|\mathbf{v}'\|^2} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ -1/\sqrt{6} & -1/\sqrt{6} & -2/\sqrt{6} \\ 1/\sqrt{2} & 1/\sqrt{2} & 0 \end{vmatrix} = (1/3, -1/3, 1/3),$$

где $\mathbf{p}'_0$ является радиус вектором точки оси, ближайшей к началу координат O . Полученный результат является точкой через которую будет проходить прямая $\mathbf{e}'$.

2.6.2. Пример №2

Рассмотрим прямую, проходящую через точку $O = (0, 0, 1)^T$ с направляющим вектором:

$$\mathbf{v} = (2, 1, 0)^T.$$

Вычислим момент:

$$\mathbf{m} = \mathbf{p} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 0 & 0 & 1 \\ 2 & 1 & 0 \end{vmatrix} = \begin{bmatrix} -1 \\ 2 \\ 0 \end{bmatrix}.$$

Соответствующий винт:

$$\mathbf{e} = \mathbf{v} + \varepsilon \mathbf{m} = (2, 1, 0)^T + \varepsilon(-1, 2, 0)^T.$$

Зададим дуальный угол поворота $A = \pi/2 + \varepsilon$. Применим формулу Родрига с $\mathbf{E} = (0, 0, 1)^T$:

$$\begin{aligned} \mathbf{e}' &= \cos A (2, 1, 0) + \sin A \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 0 & 0 & 1 \\ 2 - \varepsilon & 1 + 2\varepsilon & 0 \end{vmatrix} = -\varepsilon(2, 1, 0) + (-1 - 2\varepsilon, 2 - \varepsilon, 0) = \\ &= \begin{bmatrix} -1 - 4\varepsilon \\ 2 - 2\varepsilon \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \\ 0 \end{bmatrix} + \varepsilon \begin{bmatrix} -4 \\ -2 \\ 0 \end{bmatrix}. \end{aligned}$$

Нормализуем исходный вектор:

$$\hat{\mathbf{v}} = \left(\frac{2}{\sqrt{5}}, \frac{1}{\sqrt{5}}, 0 \right)^T, \quad \hat{\mathbf{m}} = \left(-\frac{1}{\sqrt{5}}, \frac{2}{\sqrt{5}}, 0 \right)^T.$$

Найдем точку на преобразованной оси:

$$\mathbf{p}'_0 = \frac{\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ -1 & 2 & 0 \\ -4 & -2 & 0 \end{vmatrix}}{5} = \frac{(0, 0, 10)^T}{5} = (0, 0, 2)^T.$$

2.6.3. Программная реализация на Python

Взяв за основу вычисленный ранее пример (предложенный в разделе 2.6.1), проведем аналогичные вычисления, но в программном виде. Для обработки дуальных чисел необходимо использовать следующий модуль [1], реализующий операции над дуальными числами.

Задав начальные параметры примера, вычислим момент и винт:

```

1
2 m = np.cross(p, v)
3 # Действительную часть берем из вектора v, а дуальную часть из
  ↪ момента m
4 L = np.array([Dual(v[i], m[i]) for i in range(3)], dtype=Dual)
5 # Теперь L – винт, задающий прямую
6

```

Теперь введем дуальный угол и проведем вращение вокруг оси с помощью формулы Родрига:

```

1
2 def Rodrig(A, Θ, L):

```

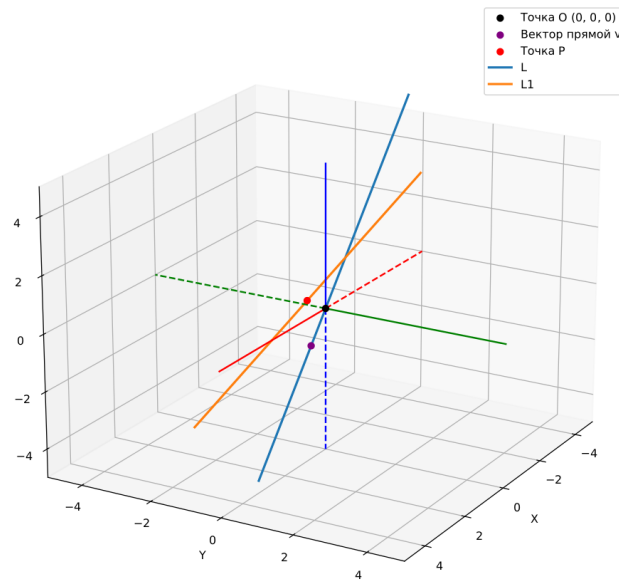


Рис. 2.1.: Реализация винтового движения вокруг оси

```

3  return A * (Dual(1, 0) - Dual.cos(Θ)) * np.dot(A, L) + L *
    ↪ Dual.cos(Θ) + np.cross(A, L) * Dual.sin(Θ)
4
5  Θ = Dual(math.pi / 2, 1)  # дуальный угол
6
7  # Вычисление нового винта
8  L1 = Rodrig(A, Θ, L)
9

```

После нормализации $\mathbf{m}'$ и $\mathbf{v}'$ мы можем найти точку через которую будет проходить прямая:

```

1
2  P = np.cross(V, M) / np.linalg.norm(V)**2
3

```

Итоговый вывод программы будет выглядеть, как показано на рисунке 2.1

3. Кватернионы

3.1. Операции над кватернионами

В рамках истории, развитие чисел шло по пути увеличения размерности или как еще упоминается в источнике [9] — «ранга»:

- Действительные числа имеют размерность равную единице, поле является коммутативным и есть обратные.
- Комплексные числа имеют размерность равную двум и само поле обладает аналогичными свойствами, что и действительные числа.
- Кватернионы же не являются полем, а некоммутативным телом.
- Еще менее известной структурой являются октавы или как их еще называют октонионы, имеющие размерность равную восьми.

Заметим, что когда мы переходим от размерности двух к размерности четырех, теряется коммутативность умножения, но для любого ненулевого числа сохраняется существование обратного элемента. Например октонионы или в целом системы, обладающие размерностью больше четырех, более не обладают ассоциативностью.

Есть основное соотношение в алгебре кватернионов [6]:

$$i^2 = j^2 = k^2 = ijk = -\sigma,$$

здесь, как и ранее упоминалось, σ — скалярная единица. В других источниках может обозначаться как просто -1 . Данное соотношение, также называют правилом Гамильтона.

Сам кватернион можно задать двумя способами. Первый способ заключается в том, что мы представляем кватернион как упорядоченную четверку действительных чисел:

$$q = (q_0, q_1, q_2, q_3), \quad q_i \in \mathbb{R}.$$

Или можно задать как сумму скаляра и вектора:

$$q = q_0\sigma + \mathbf{q} = q_0\sigma + q_1i + q_2j + q_3k.$$

С точки зрения линейной алгебры такая сумма будет выглядеть странно, но именно благодаря такой сумме кватернионы применимы для вращений.

К основным операциям над кватернионами относят:

1. Два кватерниона вида $p = p_0\sigma + p_1i + p_2j + p_3k$ и $q = q_0\sigma + q_1i + q_2j + q_3k$ равны только, когда равны их компоненты:

$$p_0 = q_0, \quad p_1 = q_1, \quad p_2 = q_2, \quad p_3 = q_3.$$

2. Сложение выполняется покомпонентно:

$$p + q = (p_0 + q_0)\sigma + (p_1 + q_1)i + (p_2 + q_2)j + (p_3 + q_3)k. \quad (3.1)$$

3. Умножение кватерниона на действительное число c :

$$q = (cq_0)\sigma + (cq_1)i + (cq_2)j + (cq_3)k.$$

4. Норма кватерниона:

$$|q| = \sqrt{q^*q} = \sqrt{qq^*}. \quad (3.2)$$

5. Обратный кватернион:

$$q^{-1} = \frac{q^*}{|q|^2}. \quad (3.3)$$

Также отдельно хочу отметить сложение кватерниона, которое выполняется по формуле (3.1). Здесь можно выделить ряд свойств, которые аналогичны свойствам действительных чисел:

- Замкнутость — в результате суммы двух кватернионов получается снова кватернион.
- Нулевой элемент — $0 = 0\sigma + 0i + 0j + 0k$.
- Противоположный элемент — $-q = -q_0\sigma - q_1i - q_2j - q_3k$.
- Ассоциативность — $(p + q) + r = p + (q + r)$.
- Коммутативность — $p + q = q + p$.

Теперь рассмотрим более сложные операции над кватернионами. Например умножение двух кватернионов. Такое умножение должно удовлетворять соотношениям Гамильтона:

Произведение	Результат	Произведение	Результат
i^2	$-\sigma$	j^2	$-\sigma$
k^2	$-\sigma$	ijk	$-\sigma$
ij	k	ji	$-k$
jk	i	kj	$-i$
ki	j	ik	$-j$

Отметим также, что умножение не является коммутативным, то есть нам важен порядок умножения, но существуют условия коммутативности, описанные в [11]. Два кватерниона p и q коммутируют ($pq = qp$) тогда и только тогда, когда выполняется хотя бы одно из условий:

1. Если векторная часть одного из кватернионов равна 0, то есть или $p = 0$, или $q = 0$, такой кватернион является скаляром.
2. У рассматриваемых кватернионов векторные части пропорциональны: $p = \alpha q$ для некоторого $\alpha \in \mathbb{R}$.

Помимо умножения на действительное число, также есть общая формула умножения. Пусть $p = p_0\sigma + p_1i + p_2j + p_3k$ и $q = q_0\sigma + q_1i + q_2j + q_3k$, тогда с учетом таблицы для умножения базисных элементов получаем:

$$pq = (p_0\sigma + p_1i + p_2j + p_3k)(q_0\sigma + q_1i + q_2j + q_3k) = p_0q_0\sigma^2 + p_0q_1\sigma i + p_0q_2\sigma j + p_0q_3\sigma k + p_1q_0i\sigma + p_1q_1i^2 + p_1q_2ij + p_1q_3ik + p_2q_0j\sigma + p_2q_1ji + p_2q_2j^2 + p_2q_3jk + p_3q_0k\sigma + p_3q_1ki + p_3q_2kj + p_3q_3k^2.$$

Далее сгруппируем по базисным элементам:

- $p_0q_0 - p_1q_1 - p_2q_2 - p_3q_3$ — коэффициент при σ ;
- $p_0q_1 + p_1q_0 + p_2q_3 - p_3q_2$ — коэффициент при i ;
- $p_0q_2 - p_1q_3 + p_2q_0 + p_3q_1$ — коэффициент при j ;
- $p_0q_3 + p_1q_2 - p_2q_1 + p_3q_0$ — коэффициент при k .

Есть более компактная формула умножения через скалярное и векторное умножения:

$$pq = (p_0q_0 - (\mathbf{p}, \mathbf{q}))\sigma + p_0\mathbf{q} + q_0\mathbf{p} + \mathbf{p} \times \mathbf{q},$$

где $(\mathbf{p}, \mathbf{q}) = p_1q_1 + p_2q_2 + p_3q_3$ — скалярное произведение, а $\mathbf{p} \times \mathbf{q} = (p_2q_3 - p_3q_2)i + (p_3q_1 - p_1q_3)j + (p_1q_2 - p_2q_1)k$ — векторное произведение. У скалярного умножения, также есть свойство инвариантности при циклической перестановке [11]:

$$\text{scal}(pqr) = \text{scal}(qrp) = \text{scal}(rpq),$$

где $\text{scal}(q) = q_0$ — скалярная часть кватерниона.

Также существует случай умножения двух чистых кватернионов. Пусть у нас есть два чистых кватерниона $p_0 = q_0 = 0$, тогда:

$$\mathbf{p}\mathbf{q} = -(\mathbf{p}, \mathbf{q})\sigma + \mathbf{p} \times \mathbf{q}.$$

Заметим также, что подобное умножение кватернионов связано с операциями векторной алгебры. Провести проверку можно с помощью базисных векторов. $ij = -1$, так как $(i, i) = 1$ и $i \times i = 0$. Аналогично и для $ij = k$, так как $(i, j) = 0$ и $i \times j = k$.

Умножение $pq = r$ можно записать в матричной форме. Если $r = r_0\sigma + r_1i + r_2j + r_3k$, то:

$$\begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \end{bmatrix} = \begin{bmatrix} p_0 & -p_1 & -p_2 & -p_3 \\ p_1 & p_0 & -p_3 & p_2 \\ p_2 & p_3 & p_0 & -p_1 \\ p_3 & -p_2 & p_1 & p_0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}.$$

К основным свойствам умножения можно отнести:

- Замкнутость — в результате произведения двух кватернионов мы получим снова кватернион.
- Ассоциативность — $(pq)r = p(qr)$, также проверяется прямым вычислением.

- Некоммутативность — в общем случае $pq \neq qp$, происходит из-за векторного произведения.
- Дистрибутивность — $p(q + r) = pq + pr$ и $(p + q)r = pr + qr$.
- Существование единицы — это подразумевает, что есть кватернион ($1 = \sigma + 0i + 0j + 0k$) со скалярной частью равной единице и векторной частью равной нулю.
- Существование обратного — это означает, что для любого ненулевого кватерниона существует q^{-1} такой, что $qq^{-1} = q^{-1}q = \sigma$.

В алгебре кватернионов также вводится понятие как сопряжение. Для кватерниона $q = q_0\sigma + \mathbf{q} = q_0\sigma + q_1i + q_2j + q_3k$ комплексно-сопряженный q^* имеет вид:

$$q^* = q_0\sigma - \mathbf{q} = q_0\sigma - q_1i - q_2j - q_3k.$$

У сопряжения также есть свой ряд свойств:

- $(p + q)^* = p^* + q^*$ — сопряжение суммы;
- $(pq)^* = q^*p^*$ — сопряжение произведения, отметим, что поржок меняется;
- $q + q^* = 2q_0\sigma$ — сумма с сопряженным;
- $(q^*)^* = q$ — дважды сопряженный.

Ранее уже была приведена формула нормы кватерниона (3.2), которая обозначается как $N(q)$ или $|q|$, рассмотрим вычисления подробнее:

$$|q|^2 = q^*q = (q_0\sigma - \mathbf{q})(q_0\sigma + \mathbf{q}) = q_0^2\sigma^2 + q_0\mathbf{q} - q_0\mathbf{q} - \mathbf{q}\mathbf{q} = q_0^2\sigma - \mathbf{q}\mathbf{q},$$

но в случае с чистым кватернионом будет верно $\mathbf{q}\mathbf{q} = -(\mathbf{q}, \mathbf{q})\sigma = -|\mathbf{q}|^2\sigma$, следовательно:

$$|q|^2 = q_0^2\sigma + |\mathbf{q}|^2\sigma = (q_0^2 + q_1^2 + q_2^2 + q_3^2)\sigma.$$

Таким образом мы приходим к формуле вида (3.2).

Отталкиваясь от понятия нормы кватерниона, можно перейти к тому какие кватернионы называют единичными или нормированными. Кватернион будет являться таковым, если $|q| = 1$, тогда нужно заметить, что все компоненты по модулю не превосходят 1. Также существует норма произведения, обладающая свойством:

$$|pq| = |p||q|.$$

Для того, чтобы придти к следствию данного произведения необходимо доказательство:

$$|pq|^2 = (pq)(pq)^* = pq(q^*p^*) = p(qq^*)p^* = p|q|^2p^* = pp^*|q|^2 = |p|^2|q|^2.$$

Само следствие заключается в том, что в результате произведения двух единичных кватернионов получается снова единичный кватернион.

Обратный кватернион q^{-1} определяется по формуле (3.3), но также существует условие существования такого кватерниона:

$$qq^{-1} = q^{-1}q = \sigma.$$

Сама же формула обратного кватерниона (3.3) получается из определения нормы $qq^* = |q|^2\sigma$. Но существует и частный случай с единичным кватернионом. Если $|q| = 1$, то:

$$q^{-1} = q^*,$$

что аналогично ортогональным матрицам, где $A^{-1} = A^T$.

При работе с обратными кватернионами также важно помнить о порядке умножения так как они некоммутативны:

$$(pq)^{-1} = q^{-1}p^{-1},$$

потому что $(pq)(q^{-1}p^{-1}) = p(qq^{-1})p^{-1} = pp^{-1} = \sigma$.

3.2. Пространственные перемещения с помощью кватернионов

Ранее при описании операций над кватернионами уже рассматривались различные операции умножения с кватернионами, но не было до конца ясно, что именно происходит, когда один кватернион умножается на другой. Вращение в 4-мерном пространстве происходит в результате умножения на единичный кватернион $q = \cos \theta + \sin \theta \mathbf{n}$. Для того, чтобы понять как именно это происходит необходимо разложить 4-мерное пространство на две взаимно перпендикулярные плоскости и посмотреть, что происходит в каждой из них.

Для любого единичного вектора $\mathbf{n}$ или если точнее чистого кватерниона с $\|\mathbf{n}\| = 1$ будет верно:

$$\mathbf{n}^2 = -\sigma.$$

Это происходит, потому что для чистого кватерниона выполняется:

$$\mathbf{n}\mathbf{n} = -(\mathbf{n}, \mathbf{n})\sigma + \mathbf{n} \times \mathbf{n} = -1 \cdot \sigma + 0 = -\sigma.$$

В плоскости $\sigma, \mathbf{n}$ умножение на $(\mathbf{n}, \theta)$ будет аналогично умножению на $e^{i\theta}$ в комплексной плоскости, это является поворотом на угол θ . Если p лежит в плоскости $\sigma, \mathbf{n}$, то $q(\mathbf{n}, \theta)p$ поворот p на θ и $pq(\mathbf{n}, \theta)$ также является поворотом на θ , потому что в данной плоскости умножение коммутативно.

Любой вектор $\mathbf{n}$ перпендикулярный плоскости $\sigma, \mathbf{n}$, т.е. $(\mathbf{v}, \sigma) = 0$ и $(\mathbf{v}, \mathbf{n}) = 0$ будет верно, что $q(\mathbf{n}, \theta)\mathbf{v}$ останется в плоскости перпендикулярной $\sigma, \mathbf{n}$, но и в свою очередь $\mathbf{v}q(\mathbf{n}, \theta)$ также остается в этой плоскости. Это происходит, потому что умножение на единичный кватернион сохраняет углы, поэтому и ортогональность тоже сохранится.

Рассмотрим более подробно вращение в плоскости $\sigma, \mathbf{n}$. Пусть $\mathbf{n}$ является единичным вектором, p кватернионов в плоскости $\sigma, \mathbf{n}$, т.е. $p = a\sigma + b\mathbf{n}$, тогда получатся следующие повороты:

1. $L_q(p) = qp$ совершает поворот p на угол θ в плоскости $\sigma, \mathbf{n}$.
2. $R_q(p) = pq$ совершает поворот p на угол θ в плоскости $\sigma, \mathbf{n}$.
3. $R_{q^*}(p) = pq^*$ совершает поворот p на угол $-\theta$ в плоскости $\sigma, \mathbf{n}$.

Это доказывается тем, что $q^* = q(\mathbf{n}, -\theta)$ и свойством аддитивности угла:

$$q(\mathbf{n}, \theta)q(\mathbf{n}, \phi) = q(\mathbf{n}, \theta + \phi)$$

Если же мы хотим рассмотреть вращение в плоскости, перпендикулярной $\sigma, \mathbf{n}$, то нам необходим вектор $\mathbf{n}$, перпендикулярный нашей плоскости, т.е. мы берем чистый кватернион, который ортогонален $\mathbf{n}$, следовательно мы получим:

1. $L_q(\mathbf{v}) = q\mathbf{v}$ совершает поворот $\mathbf{v}$ на угол θ в перпендикулярной плоскости.
2. $R_q(\mathbf{v}) = \mathbf{v}q$ совершает поворот $\mathbf{v}$ на угол $-\theta$ в перпендикулярной плоскости.
3. $R_{q^*}(\mathbf{v}) = \mathbf{v}q^*$ совершает поворот $\mathbf{v}$ на угол θ в перпендикулярной плоскости.

Отметим, что направление вращения зависит от того, как именно мы умножаем, справа или слева.

Также в контексте пространственных перемещений существует ключевое тождество. Для любого комплексного числа $g = \gamma\sigma + \delta\mathbf{n}$ в плоскости $\sigma, \mathbf{n}$ будет верно:

$$\mathbf{v}g = g^*\mathbf{v}.$$

Для описания простых вращений используют сэндвич-операторы. Существуют основные используемые операторы такие, как оператор левого умножения $L_q(\rho) = q\rho$ и оператор правого умножения $R_q(\rho) = \rho q$. Для сэндвич-оператора с q с двух сторон будет верно $T_q(\rho) = q\rho q$, а для оператора с сопряжением $S_q(\rho) = q\rho q^*$. Обозначения L_q и R_q — это линейные преобразования (из-за дистрибутивности). В свою очередь T_q и S_q тоже являются линейными, но как композиции линейных.

Ранее мы рассматривали простую плоскость и перпендикулярную, сэндвич-операторы будут применимы и здесь. Для p в плоскости $\sigma, \mathbf{n}$ произойдет следующее:

1. $T_q(p) = qpq$ совершает поворот p на угол 2θ в плоскости $\sigma, \mathbf{n}$.
2. $S_q(p) = qpq^*$ является тождественным преобразованием и поэтому оставляет p неизменным.

В перпендикулярной же плоскости $\sigma, \mathbf{n}$ для $\mathbf{v}$ преобразования получатся будут такими:

1. $T_q(\mathbf{v}) = q\mathbf{v}q$ также является тождественным преобразованием.
2. $S_q(\mathbf{v}) = q\mathbf{v}q^*$ совершает поворот $\mathbf{v}$ на угол 2θ .

В результате, сэндвич-оператор совершает вращение так, что векторная часть остается всегда неизменной.

Существует также классификация подобных преобразований в пространстве. Аффинное сохраняет массу, обозначается как S_q , а полуаффинное сохраняет же нулевую массу и обозначается, как T_N . Также еще отмечают проективное, оно не сохраняет массу обозначается, как $T_{q(\mathbf{n}, \theta)}$ при $\theta \neq \pi/2$. Разные операторы дают разные типы преобразований и нужно точно знать какой оператор использовать и для чего.

Начнем рассматривать различные преобразования с вращения. Сама задача может выглядеть, как то, что требуется повернуть некоторую точку P или вектор $\mathbf{v}$ на угол θ вокруг оси L , которая к тому же будет проходить в направлении единичного вектора $\mathbf{n}$. Любой вектор $\mathbf{v}$ можно разложить на две составляющие:

$$\mathbf{v} = \mathbf{v}_{\parallel} + \mathbf{v}_{\perp},$$

где $\mathbf{v}_{\parallel}$ является проекцией на ось вращения, а $\mathbf{v}_{\perp}$ перпендикулярная составляющая. Само вращение будет происходить, как то, что параллельная часть остается неизменной, а перпендикулярная совершает поворот в своей плоскости:

$$\mathbf{v}_{\perp} = \cos \theta \mathbf{v}_{\perp} + \sin \theta (\mathbf{n} \times \mathbf{v}_{\perp}).$$

Точка $P = O + (P - O)$ при преобразовании не теряет свое начало координат.

Вернемся к использованию сэндвич-операторов. Для единичного вектора $\mathbf{n}$ и вектора $\mathbf{v}$ используем оператор с сопряжением:

$$S_{q(\mathbf{n}, \theta/2)}(\mathbf{v}) = q(\mathbf{n}, \theta/2) \mathbf{v} q^*(\mathbf{n}, \theta/2),$$

здесь он поворачивает $\mathbf{v}$ на угол θ вокруг оси $\mathbf{n}$.

Аналогично можно использовать данный сэндвич-оператор и для поворота точек в пространстве. Для точки P будет верно:

$$S_{q(\mathbf{n}, \theta/2)}(P) = q(\mathbf{n}, \theta/2) P q^*(\mathbf{n}, \theta/2),$$

здесь он поворачивает точку P на угол θ вокруг оси L .

Существует также композиция вращений. Композиция двух вращений будет равна умножению соответствующих единичных кватернионов:

$$S_q \circ S_r = S_{qr}.$$

Множество вращений создает группу, изоморфную группе единичных кватернионов по умножению с точностью до знака.

Если $q_1 = q(\mathbf{n}_1, \theta_1/2)$ и $q_2 = q(\mathbf{n}_2, \theta_2/2)$, то их произведение $q = q_1 q_2 = q(\mathbf{n}, \theta/2)$ дает вращение с параметрами:

- Косинус половинного угла:

$$\cos \frac{\theta}{2} = \cos \frac{\theta_1}{2} \cos \frac{\theta_2}{2} - \sin \frac{\theta_1}{2} \sin \frac{\theta_2}{2} (\mathbf{n}_1 \cdot \mathbf{n}_2).$$

- Ось вращения(умноженная на синус половинного угла):

$$\sin \frac{\theta}{2} \mathbf{n} = \cos \frac{\theta_2}{2} \sin \frac{\theta_1}{2} \mathbf{n}_1 + \cos \frac{\theta_1}{2} \sin \frac{\theta_2}{2} \mathbf{n}_2 + \sin \frac{\theta_1}{2} \sin \frac{\theta_2}{2} (\mathbf{n}_1 \times \mathbf{n}_2).$$

- Случай, если оси совпадают $\mathbf{n}_1 = \mathbf{n}_2 = \mathbf{n}$, то углы просто складываются:

$$q(\mathbf{n}, \theta_1/2)q(\mathbf{n}, \theta_2/2) = q(\mathbf{n}, (\theta_1 + \theta_2)/2).$$

Помимо вращения существует отражение объекта в пространстве. Например, требуется отразить точку P или вектор $\mathbf{v}$ относительно плоскости S , которая также проходит через начало координат O и перпендикулярна единичному вектору $\mathbf{n}$ (нормали).

Как и для вращения можно разложить вектор $\mathbf{v} = \mathbf{v}_{\parallel} + \mathbf{v}_{\perp}$, здесь $\mathbf{v}_{\parallel}$ является частью параллельную нормали (перпендикулярна плоскости), а $\mathbf{v}_{\perp}$ часть, лежащая в плоскости отражения. После того, как отражение произошло, сама параллельная часть меняет знак на противоположный. Перпендикулярная останется неизменной.

Для точки ничего не меняется.

Существует, как и в случае с вращением, кватернионное представление. Оператор $T_{q(\mathbf{n}, \theta)}$ является тождественным на векторах, которые перпендикулярны $\mathbf{n}$ и поворачивает векторы в плоскости σ , $\mathbf{n}$ на угол 2θ . Чтобы получить отражение, нужно, чтобы $\mathbf{v}_{\parallel}$ сменил свой знак, а $\mathbf{v}_{\perp}$ остался неизменным. Это будет соответствовать $2\theta = \pi$, т.е. $\theta = \pi/2$. Но $q(\mathbf{n}, \pi/2) = \cos(\pi/2)\sigma + \sin(\pi/2)\mathbf{n} = \mathbf{n}$.

Для того, чтобы отразить векторы в пространстве используем сэндвич-оператор с единичным вектором. Для единичного вектора $\mathbf{n}$ и вектора $\mathbf{v}$ будет верно:

$$T_{\mathbf{n}}(\mathbf{v}) = \mathbf{n}\mathbf{v}\mathbf{n},$$

где оператор дает отражение $\mathbf{v}$ относительно плоскости, которая перпендикулярна $\mathbf{n}$.

В случае с отражениями также существует композиция. Композиция двух отражений эквивалентна одному вращению:

$$T_{\mathbf{n}_2} \circ T_{\mathbf{n}_1} = S_{\mathbf{n}_2\mathbf{n}_1},$$

но существует и обратное утверждение, что любое вращение может быть представлено как композиция двух отражений.

Если взять два единичных вектора $\mathbf{u}, \mathbf{v}$ в плоскости, которая перпендикулярна $\mathbf{n}$, таких что угол между ними равен $\theta/2$, то;

$$-\mathbf{v}\mathbf{u} = \cos(\theta/2)\sigma + \sin(\theta/2)\mathbf{n} = q(\mathbf{n}, \theta/2),$$

$$S_{q(\mathbf{n}, \theta/2)} = T_{\mathbf{v}} \circ T_{\mathbf{u}}.$$

Заметим, что сэндвич-оператор с $\mathbf{n}$ не дает отражение точки. Для точки P :

$$T_{\mathbf{n}}(P) = \mathbf{n}P\mathbf{n},$$

где сэндвич-оператор поворачивает точку на угол π вокруг оси $\mathbf{n}$, а не отражается относительно плоскости. Но возникает тогда вопрос, как же правильно получить отражение точки? Чтобы получить отражение точки P относительно плоскости через O перпендикулярно $\mathbf{n}$, нужно применять сэндвич-оператор к $P - 2\sigma$:

$$T_{\mathbf{n}}(P - 2\sigma) = \mathbf{n}(P - 2\sigma)\mathbf{n}.$$

Еще не менее важная тема в контексте преобразований это проекция. Например, мы хотим спроецировать точку P из некоторой точки обзора E на плоскость проекции S , которая проходит через точку перпендикулярную единичному вектору нормали $\mathbf{n}$. В данном случае мы можем использовать оператор $T_{q(\mathbf{n}, -\pi/4)}$. Использование $\theta = -\pi/4$ является наиболее подходящим, потому что $T_{q(\mathbf{n}, -\pi/4)}$ вращает плоскость σ , $\mathbf{n}$ на угол $2(-\pi/4) = -\pi/2$, а это значит, что единичный вектор $\mathbf{n}$ переходит в точку σ , т.е. начало координат. Начало координат же в свою очередь переходит в вектор $\mathbf{n}$. Таким образом, оператор как бы преобразует расстояние вдоль нормали в массу в начале координат.

Рассмотрим ортогональные матрицы и их связь с перспективной проекцией. Предположим, что точка проекции на оси Oz будет $E = (0, 0, 1)$, плоскость проекции будет XY -плоскостью, т.е. $z = 0$, а нормаль $\mathbf{n} = (0, 0, -1)$. Для точки $P = (x, y, z)$ формула перспективной проекции будет выглядеть следующим образом:

$$G = \left(\frac{x}{1-z}, \frac{y}{1-z}, 0 \right).$$

Сама матрица проекции 4×4 :

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Применим теперь кватернионный подход, сэндвич-оператор $T_{q(\mathbf{n}, -\pi/4)}$ является вращением в 4-мерном пространстве в плоскости σ , $\mathbf{n}$. В нашем случае плоскость σ , $\mathbf{n}$ — это плоскость $w(-z)$, т.е. вращение на $-\pi/2$ в плоскости zw . Тогда соответствующая ортогональная матрица:

$$R = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Перспективную проекцию можно реализовать как вращение в 4-мерном пространстве с последующим ортогональным проецированием.

В контексте применения к точкам, $T_{q(\mathbf{n}, -\pi/4)}(P)$ не дает перспективную проекцию точки. Оператор не является аффинным, он переводит начало координат в вектор:

$$T_{q(\mathbf{n}, -\pi/4)}(\sigma) = -\mathbf{n} \neq \sigma,$$

но $T_{q(\mathbf{n}, -\pi/4)}(P) - T_{q(\mathbf{n}, -\pi/4)}(E) = T_{q(\mathbf{n}, -\pi/4)}(P - E)$, поэтому с помощью вычитания точечной массы, мы все еще можем получить проекцию.

Также отметим, что перенос и проекция коммутируют. Проецирование точки из точки проекции E на плоскость S с последующим переносом на вектор $\mathbf{v}$ является эквивалентным сначала переносу всего на $\mathbf{v}$, а затем проецированию из перенесенной точки проекции $E + \mathbf{v}$ на перенесенную плоскость $S + \mathbf{v}$.

Ранее были изучены уже два сэндвич-оператора $S_q(\rho) = q\rho q^*$, которое является вращением в 3-мерном пространстве и $T_q(\rho) = q\rho q$ отражение (при $q = \mathbf{n}$)

или перспективная проекция (при $q = q(\mathbf{n}, -\theta/2)$). Но мы не рассматривали более подробно простые умножения вроде левого $L_q(\rho) = q\rho$ и правого $R_q(\rho) = \rho q$. Они представляют собой двойные вращения в 4-мерном пространстве и вращают одновременно в обеих плоскостях. В 3-мерном пространстве это будет давать комбинацию вращения и проекции.

Пусть $r = q(\mathbf{n}, \theta/2)$, тогда $r^2 = q$ и $r^* = r^{-1}$. Отметим также, что в источнике [6] приведено разложение вида $q\rho = r(r\rho r^*) = T_r(S_r(\rho))$, следовательно:

$$L_q(\rho) = T_r(S_r(\rho)) = S_r(T_r(\rho)).$$

В общем случае они не коммутируют, но в данном да, потому что r коммутирует с $S_r(\rho)$ в плоскости $\sigma, \mathbf{n}$, тогда можно сделать вывод, что левое умножение на q является комбинацией вращения S_r и перспективной проекции T_r . Такие преобразования и называются роторперспективами.

Есть различные частные случаи, например, когда $\theta = \pi/2$. Тогда $q = \mathbf{n}$ и $r = q(\mathbf{n}, \pi/4)$. Сначала нужно повернуть вектор $\mathbf{v}$ на угол $\pi/2$ вокруг оси $\mathbf{n}$

$$S_r(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{n})\mathbf{n} + \mathbf{n} \times \mathbf{v},$$

а затем спроецировать полученную точку $\sigma + S_r(\mathbf{v})$ из точки проекции $E = \sigma + \mathbf{n}$ на плоскость через σ , перпендикулярную $\mathbf{n}$:

$$T_r(S_r(\mathbf{v})) = -(\mathbf{n} \cdot \mathbf{v})\sigma + \mathbf{n} \times \mathbf{v} = \mathbf{n}\mathbf{v}.$$

Таким образом, произведение двух векторов $\mathbf{n}\mathbf{v}$ и есть роторперспектива.

Есть случай и при $\theta = \pi, q = -\sigma, r = q(\mathbf{n}, \pi/2) = \mathbf{n}$, тогда S_r является вращением на π вокруг $\mathbf{n}$, а $T_r = T_{\mathbf{n}}$ отражением в плоскости, перпендикулярной $\mathbf{n}$. Комбинация вращения и отражения называется ротоотражением.

4. Бикватернионы

Бикватернион — это гиперкомплексное число, которое имеет два основных способа задания. Для начала рассмотрим первый способ задать бикватернион, с помощью кватерниона, то есть у нас будут главная и моментная части. Записывается это достаточно просто, как сумма двух кватернионов, где второй умножается на дуальную единицу, например ε :

$$\Lambda = \lambda + \varepsilon\lambda^o,$$

как было ранее отмечено, ε — дуальная единица или ее еще называют символом Клиффорда [8]. Обычным кватернионом является λ , он отвечает за вращение и является главной частью:

$$\lambda = \lambda_0\sigma + \lambda_1i + \lambda_2j + \lambda_3k.$$

В свою очередь λ^o является моментной частью, это тоже кватернион, но он умножается на дуальную единицу и отвечает за трансляцию или проще говоря перенос:

$$\lambda^o = \lambda_0^o\sigma + \lambda_1^oi + \lambda_2^oj + \lambda_3^ok$$

Символ σ присутствует здесь для обозначения начала координат или скалярной части.

Второе представление заключается в дуальном виде, то есть бикватернион состоит из скалярной и винтовой частей. Записывается как кватернион, но рассмотрев подробнее базисные элементы можно заметить, что они не действительные, а дуальные числа:

$$\Lambda = \Lambda_0\sigma + \Lambda_1i + \Lambda_2j + \Lambda_3k,$$

здесь $\Lambda_j = \lambda_j + \varepsilon\lambda_j^o$ — дуальные числа, а $\Lambda_1i + \Lambda_2j + \Lambda_3k$ — винтовая часть кватерниона.

Основное отличие второго представления от первого в том, что здесь будут работать все формулы алгебры кватернионов, достаточно произвести замену действительных чисел на дуальные [10]. Упомянув замену одного на другое можно подойти к сути принципа Котельникова-Штуди. Следуя этому принципу действительные числа заменяются на дуальные, векторы на винты, кватернионное сопряжение на бикватернионное.

4.1. Операции над бикватернионами

Предположим у нас есть два бикватерниона, тогда для них будет верно:

$$\Lambda = \lambda + \varepsilon\lambda^o = (\lambda_0\sigma + \lambda_1i + \lambda_2j + \lambda_3k) + \varepsilon(\lambda_0^o\sigma + \lambda_1^oi + \lambda_2^oj + \lambda_3^ok),$$

$$M = \mu + \varepsilon\mu^o = (\mu_0\sigma + \mu_1i + \mu_2j + \mu_3k) + \varepsilon(\mu_0^o\sigma + \mu_1^oi + \mu_2^oj + \mu_3^ok).$$

Если рассматривать операцию сложения подробно, то она выполняется покомпонентно, иначе говоря, отдельно складываются главные части и отдельно моментные:

$$\Lambda + M = (\lambda + \mu) + \varepsilon(\lambda^o + \mu^o).$$

С дуальными числами это примет следующий вид:

$$\Lambda + M = (\Lambda_0 + M_0)\sigma + (\Lambda_1 + M_1)i + (\Lambda_2 + M_2)j + (\Lambda_3 + M_3)k,$$

где $\Lambda_j = \lambda_j + \varepsilon\lambda_j^o$, $M_j = \mu_j + \varepsilon\mu_j^o$. В качестве основных свойств такого сложения можно выделить коммутативность $\Lambda + M = M + \Lambda$ и ассоциативность $\Lambda + (M + N) = (\Lambda + M) + N$.

Аналогичным образом определяется и разность:

$$\Lambda - M = (\lambda - \mu) + \varepsilon(\lambda^o - \mu^o).$$

Если рассматривать дуальную форму бикватерниона $\Lambda = \Lambda_0\sigma + \Lambda_1i + \Lambda_2j + \Lambda_3k$, то он имеет скалярную и винтовую части. Скалярная обозначается часто, как Λ_s :

$$\Lambda_s = \Lambda_0\sigma = (\lambda_0 + \varepsilon\lambda_0^o)\sigma,$$

Винтовая же часть обозначается зачастую, как Λ_{sc} и имеет вид:

$$\Lambda_{sc} = \Lambda_1i + \Lambda_2j + \Lambda_3k = (\lambda_1 + \varepsilon\lambda_1^o)i + (\lambda_2 + \varepsilon\lambda_2^o)j + (\lambda_3 + \varepsilon\lambda_3^o)k.$$

Это винт(дуальный вектор), он содержит в себе направление и момент. Тогда будет верным следующее выражение в качестве полной записи:

$$\Lambda = \Lambda_s + \Lambda_{sc}.$$

Как ранее и упоминалось умножение бикватернионов будет выполняться по аналогии с умножением кватернионов, но с учетом того, что коэффициентами должны являться дуальные числа. Предположим, что $\Lambda = \Lambda_0\sigma + \Lambda_1i + \Lambda_2j + \Lambda_3k$ и $M = M_0\sigma + M_1i + M_2j + M_3k$, где Λ_j, M_j — дуальные числа. Их произведение можно получить перемножением сумм с учетом базисных элементов:

$$\begin{aligned} \Lambda \circ M = & (\Lambda_0M_0 - \Lambda_1M_1 - \Lambda_2M_2 - \Lambda_3M_3)\sigma + (\Lambda_0M_1 + \Lambda_1M_0 + \Lambda_2M_3 - \Lambda_3M_2)i + \\ & + (\Lambda_0M_2 - \Lambda_1M_3 + \Lambda_2M_0 + \Lambda_3M_1)j + (\Lambda_0M_3 + \Lambda_1M_2 - \Lambda_2M_1 + \Lambda_3M_0)k. \end{aligned}$$

Однако существует более компактная формула через винтовую алгебру, здесь будут использоваться скалярное и винтовое произведения винтов:

$$\Lambda \circ M = \Lambda_0M_0 - (\Lambda_{sc} \cdot M_{sc}) + \Lambda_0M_{sc} + M_0\Lambda_{sc} + \Lambda_{sc} \times M_{sc}.$$

Помимо конкретных вспомогательных формул существует несколько свойств умножения для бикватернионов:

- $\Lambda \circ M \neq M \circ \Lambda$ — некоммутативность. Также по аналогии с кватернионами это происходит из-за того, что существует антикоммутативность векторного произведения;

- $\Lambda \circ (M + N) = \Lambda \circ M + \Lambda \circ N$ и $(M + N) \circ \Lambda = M \circ \Lambda + N \circ \Lambda$ — дистрибутивность. Можно проверить вычислив напрямую;
- $\Lambda \circ (M \circ N) = (\Lambda \circ M) \circ N$ — ассоциативность.

Существует также частный случай — произведение двух винтов. Например, Λ и M — чистые винты и мы знаем, что их скалярные части равны нулю, в таком случае формула значительно упрощается:

$$\Lambda \circ M = -(\Lambda \cdot M) + \Lambda \times M.$$

Это происходит, потому что в общей формуле уходят члены $\Lambda_0 M_0$, $\Lambda_0 M_{sc}$ и $M_0 \Lambda_{sc}$, в итоге остается винтовое и скалярное произведения. Помимо произведения двух винтов есть частный случай с квадратом винта:

$$\Lambda \circ \Lambda = -(\Lambda \cdot \Lambda) + \Lambda \times \Lambda = -|\Lambda|^2 + 0 = -|\Lambda|^2,$$

так как при векторном произведении вектора на самого себя произойдет обнуление.

Бикватернионное сопряжение аналогично кватернионному. Для бикватерниона $\Lambda = \Lambda_0 \sigma + \Lambda_1 i + \Lambda_2 j + \Lambda_3 k$ будет верно, если сопряженный получится сменой знака у всех компонент вектора — $\bar{\Lambda}$:

$$\bar{\Lambda} = \Lambda_0 \sigma - \Lambda_1 i - \Lambda_2 j - \Lambda_3 k.$$

В случае с кватернионной формой, которая содержит моментную и главную части, выражение будет выглядеть следующим образом:

$$\bar{\Lambda} = \overline{\lambda + \varepsilon \lambda^o} = \lambda^* + \varepsilon (\lambda^o)^*,$$

где $\lambda^* = \lambda_0 \sigma - \lambda_1 i - \lambda_2 j - \lambda_3 k$ — обычное сопряжение кватернионов.

Опираясь на предыдущий раздел о сопряжении бикватерниона, можно теперь найти норму бикватерниона. Норма бикватерниона получается в результате произведения бикватерниона на его сопряженный бикватернион:

$$\|\Lambda\| = \Lambda \circ \bar{\Lambda} = \bar{\Lambda} \circ \Lambda.$$

Если опустить сам громоздкий вывод формулы нормы бикватерниона, то в итоге мы получим следующего вида выражение:

$$\|\Lambda\| = (\lambda_0^2 + \lambda_1^2 + \lambda_2^2 + \lambda_3^2) + 2\varepsilon(\lambda_0 \lambda_0^o + \lambda_1 \lambda_1^o + \lambda_2 \lambda_2^o + \lambda_3 \lambda_3^o),$$

также есть более компактная запись этой формулы :

$$\|\Lambda\| = |\lambda|^2 + 2\varepsilon(\lambda \cdot \lambda^o),$$

где $\lambda \cdot \lambda^o$ — скалярное произведение кватернионов.

Бикватернион Λ^{-1} называется обратным к Λ , если:

$$\Lambda \circ \Lambda^{-1} = \Lambda^{-1} \circ \Lambda = \sigma.$$

Также существует формула обратного бикватерниона:

$$\Lambda^{-1} = \|\Lambda\|^{-1} \bar{\Lambda},$$

заметим, что $\|\Lambda\|^{-1}$ — это обратное дуальное число, а оно существует не всегда.

Предположим, что у нас есть дуальное число $A = a + \varepsilon a^o$, тогда обратное будет иметь вид:

$$A^{-1} = \frac{1}{a} - \varepsilon \frac{a^o}{a^2},$$

если $a \neq 0$. Если же $a = 0$, то обратное число не может существовать в целом, потому что произойдет деление на ноль в главной части. Тогда подведем некоторый итог, для нормы $\|\Lambda\| = |\lambda|^2 + 2\varepsilon(\lambda \cdot \lambda^o)$ обратное существует только, если главная часть $|\lambda|^2 \neq 0$. Тогда рассмотрим условие существования обратного кватерниона. Главная часть $\lambda \neq 0$. Если $\lambda = 0$, но $\lambda^o \neq 0$, то $\|\Lambda\| = 0 + 2\varepsilon(0 \cdot \lambda^o) = 0$, в таком случае обратного не существует.

Для обратного бикватерниона формула имеет вид:

$$\Lambda^{-1} = \left(\frac{1}{|\lambda|^2} - \varepsilon \frac{2(\lambda \cdot \lambda^o)}{|\lambda|^4} \right) (\lambda^* + \varepsilon(\lambda^o)^*)$$

Вспомнив свойство некоммутативности бикватернионов, мы знаем, что деление нельзя произвести за одну операцию, необходимы две разные операции.

Для начала разберемся с делением в левой части, то есть с первой операцией. У нас есть уравнение $\Lambda \circ X = M$ относительно X . нужно умножить на Λ^{-1} :

$$\Lambda^{-1} \circ (\Lambda \circ X) = (\Lambda^{-1} \circ \Lambda) \circ X = \sigma \circ X = X = \Lambda^{-1} \circ M,$$

следовательно результатом первой операции будет $X = \Lambda^{-1} \circ M$.

Перейдем теперь к делению правой части, второй операции. Решим уравнение $X \circ \Lambda = M$ относительно X . Нужно умножить на Λ^{-1} :

$$(X \circ \Lambda) \circ \Lambda^{-1} = X \circ (\Lambda \circ \Lambda^{-1}) = X \circ \sigma = X = M \circ \Lambda^{-1},$$

следовательно результатом второй операции деления будет $X = M \circ \Lambda^{-1}$.

Параметра и инварианта нет в обычной кватернионной алгебре, их появление объясняется как раз появлением дуальных частей. Инвариант бикватерниона $\Lambda = \lambda + \varepsilon \lambda^o$ является скалярным произведением главной и моментной частей:

$$\Lambda_i n v = \lambda \cdot \lambda^o = \lambda_0 \lambda_0^o + \lambda_1 \lambda_1^o + \lambda_2 \lambda_2^o + \lambda_3 \lambda_3^o.$$

Это называется инвариантом, потому что величина не меняется при некоторых преобразованиях.

Рассмотрев инвариант, можно подойти к параметру, потому что параметр бикватерниона p — это отношение инварианта к квадрату нормы главной части:

$$p = \frac{\lambda \cdot \lambda^o}{|\lambda|^2} = \frac{\lambda_0 \lambda_0^o + \lambda_1 \lambda_1^o + \lambda_2 \lambda_2^o + \lambda_3 \lambda_3^o}{\lambda_0^2 + \lambda_1^2 + \lambda_2^2 + \lambda_3^2},$$

отметим, что $p = \infty$ является чистым моментом, потому что главная часть обнуляется, а $p = 0$ соответствует чистой прямой.

Выделим основные частные случаи:

1. Для скалярной части, где винтовая часть $\Lambda_{sc} = 0$:

$$p_s = \frac{\lambda_0^o}{\lambda_0}$$

2. Для винтовой части, где скалярная часть $\Lambda_s = 0$:

$$p_{sc} = \frac{\lambda_1 \lambda_1^o + \lambda_2 \lambda_2^o + \lambda_3 \lambda_3^o}{\lambda_1^2 + \lambda_2^2 + \lambda_3^2}.$$

Тензором бикватерниона Λ называются квадратный корень из его нормы:

$$|\Lambda| = \sqrt{\|\Lambda\|}.$$

Для того, чтобы найти модуль мы можем использовать формулу корня из дуального числа при $|\lambda| \neq 0$:

$$|\Lambda| = \sqrt{|\lambda|^2 + 2\varepsilon(\lambda \cdot \lambda^o)} = |\lambda| \left(1 + \varepsilon \frac{\lambda \cdot \lambda^o}{|\lambda|^2} \right) = |\lambda|(1 + \varepsilon p).$$

Если главная часть $\lambda = 0$ или даже норма $\|\Lambda\| \neq 0$, то модуль теряет смысл, потому что корень из дуального числа, где главная часть нулевая не может быть определена однозначно.

В случае с верзором, то это просто единичный бикватернион, который получили с помощью нормировки:

$$\Lambda_n = |\Lambda|^{-1} \Lambda.$$

При разложении бикватерниона:

$$\Lambda = |\Lambda| \Lambda_n.$$

Значит, что любой бикватернион может быть представлен, как произведение его модуля(тензора) и верзора.

4.2. Тригонометрическое представление бикватерниона

Тригонометрическое представление позволяет обобщить форму кватерниона $q = |q|(\cos \theta + \mathbf{n} \sin \theta)$ на бикватернионы. Вернемся к дуальным числам, предположим, что у нас есть два дуальных числа $A = a + \varepsilon a^o$ и $B = b + \varepsilon b^o$, и они удовлетворяют условию $A^2 + B^2 = 1$, то существует такой дуальный угол $\Theta = \theta + \varepsilon \theta^o$, что:

$$\cos \Theta = A, \quad \sin \Theta = B.$$

Опираясь на описанное выше, применим это к бикватерниону. Опять же рассмотрим два дуальных числа $A = \frac{\Lambda_0}{|\Lambda|}$, $B = \frac{|\Lambda_{sc}|}{|\Lambda|}$, где $|\Lambda_{sc}| = \sqrt{\Lambda_1^2 + \Lambda_2^2 + \Lambda_3^2}$ — модуль винтовой части. Тогда эти числа удовлетворяют условию $A^2 + B^2 = 1$, потому что:

$$\left(\frac{\Lambda_0}{|\Lambda|} \right)^2 + \left(\frac{|\Lambda_{sc}|}{|\Lambda|} \right)^2 = \frac{\Lambda_0^2 + |\Lambda_{sc}|^2}{|\Lambda|^2} = \frac{|\Lambda|^2}{|\Lambda|^2} = 1,$$

а значит, что есть такой дуальный угол $\Theta = \theta + \varepsilon\theta^o$, что:

$$\cos \Theta = \frac{\Lambda_0}{|\Lambda|}, \quad \sin \Theta = \frac{|\Lambda_{sc}|}{|\Lambda|}.$$

Также важно ввести такое понятие, как единичный винт E , он будет задавать направление винтовой части:

$$E = \frac{\Lambda_{sc}}{|\Lambda_{sc}|} = \frac{\Lambda_1 i + \Lambda_2 j + \Lambda_3 k}{\sqrt{\Lambda_1^2 + \Lambda_2^2 + \Lambda_3^2}},$$

где E — чистый бикватернион или проще говоря винт с единичной нормой ($|E| = 1$).

Теперь бикватернион Λ можно записать в новой форме:

$$\Lambda = \Lambda_0 + \Lambda_{sc} = |\Lambda| \left(\frac{\Lambda_0}{|\Lambda|} + \frac{|\Lambda_{sc}|}{|\Lambda|} E \right) = |\Lambda| (\cos \Theta + E \sin \Theta).$$

Для единичного бикватерниона $|\Lambda| = 1$ получим следующее выражение:

$$\Lambda_n = \cos \Theta + E \sin \Theta$$

Также можно представить и обратный бикватернион в тригонометрической форме. Тут получится также как и для кватернионов, обратный получается в результате замены знака угла:

$$\Lambda^{-1} = |\Lambda|^{-1} (\cos \Theta - E \sin \Theta),$$

$$\Lambda_n^{-1} = \cos \Theta - E \sin \Theta.$$

В контексте кватернионов используется 4-мерное пространство с базисом σ, i, j, k для того, чтобы можно было представить точки и вектора, т.е. точка $P = \sigma + \mathbf{p} = \sigma + p_1 i + p_2 j + p_3 k$ и вектор $\mathbf{v} = v_1 i + v_2 j + v_3 k$. Но говоря о кватернионном представлении стоит заметить, что оно не различает типы объектов. Например будут одинаково выглядеть точка и плоскость так, как оба имеют скалярную и векторную части. Также и с обычным вектором, и вектором нормали. Тогда уже можно подойти как раз к бикватернионам, потому что они распределяют геометрические объекты по разным подпространствам 8-мерного пространства с базисом вида $\sigma, i, j, k, \sigma\varepsilon, i\varepsilon, j\varepsilon, k\varepsilon$.

Добавим некоторые пояснения к базисным элементам, используемым для представления точек и векторов:

- σ — начало координат, является скалярной единицей;
- $i\varepsilon, j\varepsilon, k\varepsilon$ — определяют направления в пространстве, содержат дуальную единицу.

Также есть такой объект, как аффинная точка. Аффинная точка — это точка P с декартовыми координатами вида (p_1, p_2, p_3) и имеет представление в виде бикватерниона:

$$P = \sigma + (p_1 i + p_2 j + p_3 k)\varepsilon = \sigma + \mathbf{p}\varepsilon.$$

Точечная масса с m и имеющей координаты $(p_1i + p_2j + p_3k)$ имеет следующее представление:

$$P = mO + (p_1i + p_2j + p_3k)\varepsilon = mO + \mathbf{p}\varepsilon.$$

Важное свойство однородности: умножение на ненулевое действительное число λ не меняет положение точки:

$$\lambda P = \lambda mO + \lambda \mathbf{p}\varepsilon \equiv \sigma + \frac{\lambda \mathbf{p}}{\lambda m} \varepsilon = \sigma + \frac{\mathbf{p}}{m} \varepsilon.$$

Перейдем теперь к векторам. Свободный вектор $\mathbf{v} = v_1i + v_2j + v_3k$, также имеет представление бикватерниона:

$$\mathbf{V} = (v_1i + v_2j + v_3k)\varepsilon = \mathbf{v}\varepsilon,$$

отметим, что в случае с векторами у нас полностью отсутствует скалярная часть.

Для представления плоскостей используются базисные элементы такие, как $\sigma\varepsilon$ — дуальная скалярная единица, которую используют для представления расстояния и i, j, k — обычные векторные единицы, являются представлением нормали.

Плоскость Π задается уравнением $(\mathbf{n}, \mathbf{x}) + d = 0$, где $\mathbf{n} = n_1i + n_2j + n_3k$ — вектор нормали, к слову, он не обязательно должен быть единичным и d — параметр, который связан с расстоянием от начала координат. Тогда плоскость представленная бикватернионом имеет следующий вид:

$$\Pi = \mathbf{n} + d\sigma\varepsilon,$$

здесь векторная часть $\mathbf{n}$ — это нормаль плоскости а дуальная скалярная часть $d\sigma\varepsilon$ — это расстояние, которое векторно умножается на направление.

Говоря о нормировке, так же как и для точек, представление плоскости является однородным:

$$\lambda \Pi = \lambda \mathbf{n} + \lambda d\sigma\varepsilon$$

задает ту же самую плоскость при любом $\lambda \neq 0$.

Для того, чтобы получить единичную нормаль $\hat{\mathbf{n}}$ и расстояние δ используют формулу:

$$\hat{\Pi} = \frac{\Pi}{|\Pi|} = \hat{\mathbf{n}} + \delta\sigma\varepsilon,$$

где $|\Pi| = \|\mathbf{n}\|$ следовательно модуль бикватерниона равен длине нормали.

Сама нормаль $\mathbf{n}$ определяет направление, оно перпендикулярно плоскости, а d относительно этой нормали определяет нахождение плоскости. Точка на плоскости, которую определяют как самую близкую к началу координат задают вектором вида:

$$-\frac{d\mathbf{n}}{\|\mathbf{n}\|^2}.$$

Существует также условие принадлежности точки к рассматриваемой нами плоскости. Используя скалярное произведение кватернионов можно убедиться в том, „лежит ли точка на плоскости или нет. Предположим, что у нас есть точка $P = \sigma + \mathbf{p}\varepsilon$ и соответственно плоскость $\Pi = \mathbf{n} + d\sigma\varepsilon$, тогда если мы хотим

получить результат скалярного произведения, то необходимо воспользоваться следующей формулой:

$$P \cdot \Pi = (\sigma + \mathbf{p}\varepsilon) \cdot (\mathbf{n} + d\sigma\varepsilon) = (d + (\mathbf{p}, \mathbf{n}))\varepsilon,$$

где само условие принадлежности заключается в том, что P лежит на $\Pi \iff P \cdot \Pi = 0 \iff d + (\mathbf{p}, \mathbf{n}) = 0$. Простыми словами результатом скалярного произведения будет дуальное число, а равенство нулю будет являться возможным только если сам коэффициент при ε будет равен нулю. То есть это в точности повторяет уравнение плоскости $(\mathbf{n}, \mathbf{p}) + d = 0$.

Для задания прямой в пространстве существует несколько способов:

- Двумя точками;
- Точкой и направляющим вектором;
- Двумя плоскостями;
- Координатами Плюккера;
- Дуальными координатами Плюккера.

Все эти способы объединяет одно условие — они требуют шесть параметров. Чаще всего конечно в пространстве бикватернионов используются координаты Плюккера.

Предположим у нас есть некоторая прямая t и она проходит через точку P , которая в свою очередь параллельна вектору $\mathbf{v}$, тогда все сводится к тому, что шесть координат Плюккера данной прямой — это пара векторов $(\mathbf{v}, \mathbf{u})$, где:

$$\mathbf{u} = (\mathbf{p} - \mathbf{o}) \times \mathbf{v} = \mathbf{p} \times \mathbf{v},$$

где $\mathbf{o} = \mathbf{0}$, потому что начало координат. В данном случае $\mathbf{v}$ — это вектор, который параллелен прямой t или иными словами направляющий вектор, а $\mathbf{u}$ — вектор нормали к плоскости, он содержит в себе прямую и начало координат. Есть также важное свойство вектора $\mathbf{u} = \mathbf{p} \times \mathbf{v}$, он не зависит от выбора точки P на прямой.

Если мы хотим найти точку на прямой и нам даны координаты Плюккера $(\mathbf{v}, \mathbf{u})$ прямой t , то есть несколько случаев. Если прямая проходит через начало координат, то $\mathbf{u} = \mathbf{0}$, а точка лежит на прямой так, как начало координат. Также может быть случай, если прямая не проходит через начало координат, тогда предположим, что у нас есть точка S . Это точка на прямой и ближайшая к началу координат, следовательно вектор $\mathbf{q} = \mathbf{OQ}$ будет перпендикулярен направляющему вектору $\mathbf{v}$. Вектор $\mathbf{u} = \mathbf{q} \times \mathbf{v}$ перпендикулярен и $\mathbf{q}$, и $\mathbf{v}$. Тогда можно утверждать, что $\mathbf{q}$, $\mathbf{v}$ и $\mathbf{u}$ взаимно ортогональны.

В векторной алгебре есть соотношение вида:

$$\mathbf{v} \times (\mathbf{q} \times \mathbf{v}) = \|\mathbf{v}\|^2 \mathbf{q},$$

тогда отсюда мы можем получить формулу, которая поможет найти ближайшую точку:

$$\mathbf{q} = \frac{\mathbf{v} \times \mathbf{u}}{\|\mathbf{v}\|^2}.$$

Соответственно, основной смысл в том, что вектор от начала координат до ближайшей искомой нами точки прямой, выражается через направляющий вектор и момент.

Так как мы уже начали рассматривать представление с помощью координат Плюккера, то существует компактная запись в виде бикватерниона:

$$L = \mathbf{v} + \mathbf{u}\varepsilon,$$

здесь $\mathbf{v}$ — вектор, параллельный прямой, а $\mathbf{u}\varepsilon$ — момент прямой.

Существует условие Плюккера, из определения $\mathbf{u} = \mathbf{p} \times \mathbf{v}$ выведено соотношение:

$$\mathbf{v} \cdot \mathbf{u} = \mathbf{v} \cdot (\mathbf{p} \times \mathbf{v}) = 0,$$

где векторы $\mathbf{v}$ и $\mathbf{u}$ ортогональные, это и есть условие Плюккера. Поэтому если мы возьмем любой бикватернион вида $L = \mathbf{v} + \mathbf{u}\varepsilon$, где $\mathbf{v} \cdot \mathbf{u} = 0$, это будет представлять некоторую прямую t в пространстве.

Для того, чтобы понять, где искать информацию о положении, можно подробнее рассмотреть отношение длин векторов. Предположим, что у нас есть расстояние от начала координат до прямой. Назовем d расстоянием от начала координат O до прямой t , следовательно для ближайшей точки S будет верно $|\mathbf{u}| = d$:

$$|\mathbf{u}| = \|\mathbf{q} \times \mathbf{v}\| = \|\mathbf{q}\| \cdot \|\mathbf{v}\| \cdot \sin 90^\circ = d \cdot \|\mathbf{v}\|,$$

тогда $\frac{\|\mathbf{u}\|}{\|\mathbf{v}\|} = d$.

Расстояние от начала координат до прямой равно отношению длин этих векторов $\mathbf{u}$ и $\mathbf{v}$. Сам вектор $\mathbf{v}$ имеет произвольную длину, но отношение $\frac{\|\mathbf{u}\|}{\|\mathbf{v}\|}$ называется инвариантом, который определяет положение прямой.

Помимо обычных координат Плюккера, как было выше сказано, есть еще дуальные. Их получают после применения операции дуализации к прямой:

$$T^\dagger = (\mathbf{v} + \mathbf{u}\varepsilon)^\dagger = \mathbf{u} + \mathbf{v}\varepsilon,$$

где $T^\dagger$ является той же самой прямой только в дуальном пространстве.

4.3. Винтовое движение с помощью бикватернионов

Вернемся к исходной формуле для кватернионов, описывающей вращение. Единичный кватернион вращения в тригонометрической форме имеет вид:

$$\lambda = \cos \frac{\theta}{2} \sigma + \sin \frac{\theta}{2} \mathbf{a},$$

где θ угол поворота, $\mathbf{a}$ единичный вектор оси вращения, а σ — скалярная единица, обозначающая начало координат. В целом скалярную единицу можно было опустить, так как у Челнокова [8] используется единица, а у Голдмана [3] вовсе O , поэтому данный символ необходим лишь для разделения скалярной и векторной частей.

Сэндвич-формула для вращения точки $P = w\sigma + \mathbf{p}$ в однородных координатах будет следующей:

$$P' = \lambda P \lambda^*.$$

Отметим, что скалярная часть w не меняется при вращении:

$$\lambda(w\sigma + \mathbf{p})\lambda^* = w\lambda\lambda^* + \lambda\mathbf{p}\lambda^* = w\sigma + \lambda\mathbf{p}\lambda^*,$$

также это будет работать и для свободных векторов, где $w = 0$.

Чтобы совершить переход от кватернионного способа вращения к бикватернионному необходимо воспользоваться принципом перенесения Котельникова-Штуди [7]. Это значит что угол θ заменяется на дуальный угол $\Theta = \theta + \varepsilon\theta^o$, ось(вектор) $\mathbf{a}$ на винт $\mathbf{A} = \mathbf{a} + \varepsilon\mathbf{a}^o$, а кватернионное сопряжение $(\cdot)^*$ на бикватернионное сопряжение $(\cdot)^\dagger$.

Применяя данный принцип перенесения Котельникова-Штуди, мы получаем бикватернион винтового движения:

$$\Lambda = \cos \frac{\Theta}{2} + \sin \frac{\Theta}{2} \mathbf{A},$$

где $\Theta = \theta + \varepsilon\theta^o$ и $\mathbf{A} = \mathbf{a} + \varepsilon\mathbf{a}^o$.

Есть более развернутая форма через действительные числа:

$$\Lambda = \cos \frac{\theta}{2} + \sin \frac{\theta}{2} (\mathbf{a} + \varepsilon\mathbf{a}^o) + \left(\cos \frac{\theta}{2} \mathbf{a} - \sin \frac{\theta}{2} \right) \frac{\theta^o}{2} \varepsilon.$$

Также возможны частные случаи вроде чистого вращения и чистой трансляции. Если $\theta^o = 0$, то это чистое вращение:

$$R = \cos \frac{\theta}{2} + \sin \frac{\theta}{2} (\mathbf{a} + \varepsilon\mathbf{a}^o).$$

Если же $\theta = 0$, то это чистая трансляция:

$$T = 1 + \frac{\theta^o}{2} \mathbf{a} \varepsilon.$$

Здесь как раз применима коммутативность, потому что для винтового движения вдоль одной оси R и T коммутируют, следовательно $RT = TR$.

Аффинная точка P может быть представлена в виде бикватерниона:

$$P = \sigma + \mathbf{p}\varepsilon = \sigma + (xi + yj + zk)\varepsilon.$$

Тогда рассмотрим сэндвич-формулу для винтового движения точки:

$$P' = \Lambda P \Lambda^\dagger,$$

а для полной формулы точки будет верно:

$$P' = \Lambda P \Lambda^\dagger = \sigma + (\cos \theta \mathbf{p} + \sin \theta \mathbf{a} \times \mathbf{p} + (1 - \cos \theta)(\mathbf{a}, \mathbf{p})\mathbf{a} + \sin \theta \mathbf{a}^o + (1 - \cos \theta) \mathbf{a} \times \mathbf{a}^o + \theta^o \mathbf{a}) \varepsilon$$

Аналогично эту формулу можно получить, используя вращение вокруг произвольной оси. Достаточно перенести начало координат в точку $\mathbf{p}_0 = \mathbf{a} \times \mathbf{a}^o$ на оси, произвести вращение по формуле Родрига и сделать обратный перенос.

Если рассматривать чистое вращение, то для свободного вектора $\mathbf{V} = \mathbf{v}\varepsilon$:

$$\mathbf{V}' = \Lambda \mathbf{V} \Lambda^\dagger = (\cos \theta \mathbf{v} + \sin \theta \mathbf{a} \times \mathbf{v} + (1 - \cos \theta)(\mathbf{a}, \mathbf{v})\mathbf{a})\varepsilon,$$

где трансляционная часть $\Lambda \Lambda^\dagger$ никак не влияет на свободный вектор.

Теперь рассмотрим чистую трансляцию точки. Для бикватерниона трансляции $T = 1 + \frac{\theta^o}{2}\mathbf{a}\varepsilon$:

$$P' = T P T^\dagger = \sigma + (\mathbf{p} + \theta^o \mathbf{a})\varepsilon.$$

В случае со свободным вектором трансляция не действует:

$$\mathbf{V}' = T \mathbf{V} T^\dagger = \mathbf{v}\varepsilon = \mathbf{V}.$$

Предположим, что прямая L задается чистым бикватернионом:

$$L = \mathbf{v} + \mathbf{m}\varepsilon.$$

Тогда можно рассмотреть сэндвич-формулу для прямой. Заметим, что в отличие от точки, для прямой используется сопряжение $(\cdot)^*$, а не $(\cdot)^\dagger$:

$$L' = \Lambda L \Lambda^*.$$

Чистое вращение прямой для $R = \cos \frac{\theta}{2} + \sin \frac{\theta}{2}(\mathbf{a} + \varepsilon \mathbf{a}^o)$:

$$L' = R L R^*,$$

если опустить громоздкие выкладки, полная формула чистого вращения будет иметь вид:

$$L' = R(\mathbf{v} + \mathbf{m}\varepsilon)R^* = \left[\cos \theta \mathbf{v} + \sin \theta \mathbf{a} \times \mathbf{v} + (1 - \cos \theta)(\mathbf{a}, \mathbf{v})\mathbf{a} \right] + \\ + \left[\cos \theta \mathbf{m} + \sin \theta \mathbf{a} \times \mathbf{m} + (1 - \cos \theta)(\mathbf{a}, \mathbf{m})\mathbf{a} + \sin \theta \mathbf{a}^o \times \mathbf{v} + (1 - \cos \theta) \mathbf{v} \times (\mathbf{a} \times \mathbf{a}^o) \right] \varepsilon.$$

Чистая трансляция для $T = 1 + \frac{\theta^o}{2}\mathbf{a}\varepsilon$:

$$L' = T L T^*,$$

тогда полный вид формулы будет иметь вид:

$$L' = \mathbf{v} + (\mathbf{m} + \theta^o \mathbf{a} \times \mathbf{v})\varepsilon$$

Плоскость Π задается уравнением $(\mathbf{n}, \mathbf{x}) + d = 0$ и может быть представлена в качестве бикватерниона:

$$\Pi = \mathbf{n} + d\varepsilon,$$

где $\mathbf{n}$ — вектор нормали, он может быть не обязательно единичным и d — параметр расстояния.

Сама сэндвич-формула для плоскости имеет вид:

$$\Pi' = \Lambda \Pi \Lambda^\dagger,$$

отметим, что здесь уже используется сопряжение $(\cdot)^\dagger$.

Чистое вращение плоскости для $R = \cos \frac{\theta}{2} + \sin \frac{\theta}{2}(\mathbf{a} + \varepsilon \mathbf{a}^o)$:

$$\Pi' = R\Pi R^\dagger = R\mathbf{n}R^\dagger + d\varepsilon R R^\dagger,$$

тогда полная формула:

$$\begin{aligned} \Pi' = R(\mathbf{n} + d\varepsilon)R^\dagger = & \left[\cos \theta \mathbf{n} + \sin \theta \mathbf{a} \times \mathbf{n} + (1 - \cos \theta)(\mathbf{n}, \mathbf{a})\mathbf{a} \right] + \\ & + \left[d - \sin \theta (\mathbf{a}^o, \mathbf{n}) - (1 - \cos \theta) (\mathbf{a}, \mathbf{n}, \mathbf{a}^o) \right] \varepsilon. \end{aligned}$$

Чистая трансляция для $T = 1 + \frac{\theta^o}{2}\mathbf{a}\varepsilon$:

$$\Pi' = T\Pi T^\dagger,$$

следовательно итоговое выражение будет иметь вид:

$$\Pi' = \mathbf{n} + (d + \theta^o(\mathbf{a}, \mathbf{n}))\varepsilon.$$

4.4. Отражение относительно плоскости

Для того, чтобы провести отражение какого-либо объекта в пространстве относительно плоскости Π необходимо задать ее единичной нормалью $\mathbf{n}$ и расстоянием d от начала координат:

$$N = \mathbf{n} + d\varepsilon.$$

Задав плоскость относительно, которой будем отражать, можно перейти к примерам таких объектов. Для отражения точки $P = w\sigma + \mathbf{p}\varepsilon$ будет верно:

$$P' = -NP^*N = \sigma + (\mathbf{p} - 2(\mathbf{p}, \mathbf{n})\mathbf{n} - 2d\mathbf{n})\varepsilon.$$

В случае с прямой $L = \mathbf{v} + \mathbf{m}\varepsilon$ используется уже другое выражение с использованием обратного элемента:

$$L' = NL^\dagger N^{-1} = \mathbf{n}\mathbf{v}\mathbf{n} + ((P' - \sigma) \times (\mathbf{n}\mathbf{v}\mathbf{n})),$$

так как $\|\mathbf{n}\| = 1$, то $N^{-1} = N^* = -\mathbf{n} + d\varepsilon$. Такой же вид формулы для отражения точки имеется и в [3].

Для плоскости $\Pi = \boldsymbol{\nu} + \delta\varepsilon$, которую отражаем и $N = \mathbf{n} + d\varepsilon$ самой плоскости отражения предлагается использовать следующее выражение:

$$\Pi' = -N\Pi^*N = \boldsymbol{\nu} - 2(\boldsymbol{\nu}, \mathbf{n})\mathbf{n} - (2d(\boldsymbol{\nu}, \mathbf{n}) - \delta)\varepsilon.$$

4.5. Практическое применение

4.5.1. Примеры винтового движения

Важно отметить, что конкретно бикватернионы не реализованы в Julia, но они активно применяются при реализации винтового движения, как в ручных вычислениях, так и в программе, предполагающей работу с винтовым движением. Благодаря перечисленным выше пакетам в Julia можно реализовать бикватернионы самостоятельно, изучив теоретические работы по бикватернионам и алгоритмам их преобразования, а затем построив собственную структуру данных и набор функций. Необходимо реализовать основные операции над бикватернионами из разделов 4 и формулы винтового движения 4.3.

Для удобства дальнейших вычислений реализуем функцию преобразования точки в бикватернион:

```
1 function point_to_dualquaternion(p::Vector)
2     return DualQuaternion(
3         Quaternion(1.0, 0.0, 0.0, 0.0),
4         Quaternion(0.0, p[1], p[2], p[3])
5     )
6 end
```

Аналогично определим функцию для извлечения координат точки из бикватерниона:

```
1 function dualquaternion_to_point(dq::DualQuaternion)
2     return [dq.qo.v1, dq.qo.v2, dq.qo.v3]
3 end
```

Теперь определим само винтовое движение, задав его основные параметры:

```
1 angle = π
2 translation_distance = 1.0
3 θ = Dual(angle, translation_distance)
```

В качестве оси винтового движения зададим ось Oz :

```
1 axis_direction = normalize([0.0, 0.0, 1.0])
2 A = DualQuaternion(
3     Quaternion(0.0, axis_direction...),
4     Quaternion(0.0, 0.0, 0.0, 0.0)
5 )
```

Для преобразования бикватерниона, соответствующего точке, используем «сэндвич-формулу»:

```
1 motor_func = PointSandwichFormula(A, θ)
```

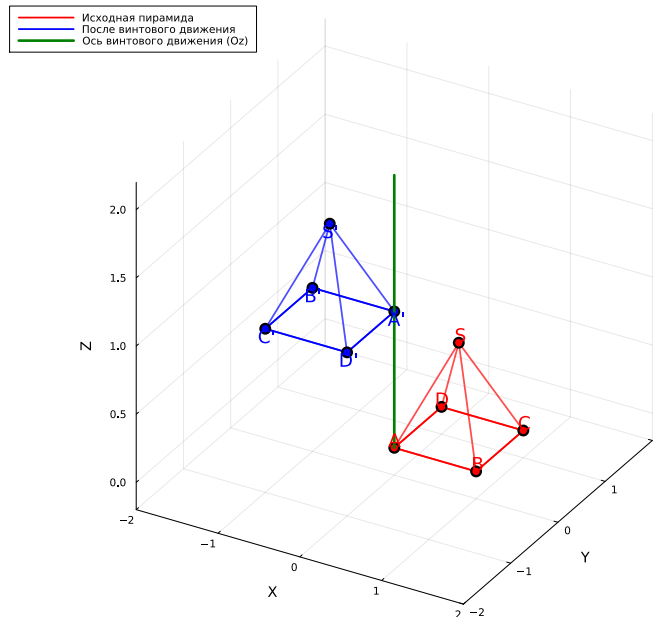


Рис. 4.1.: Винтовое движение пирамиды с параметрами вращения 180° и длиной трансляции равной единице вокруг оси Oz

Применив винтовое движение ко всем точкам пирамиды, мы получим ее отражение относительно плоскости. Итоговый результат выше описанных операций представлен на графике 4.1. Также были добавлены ползунки для дуального угла и длины трансляции с помощью пакета PlutoUI, чтобы винтовое движение объекта в пространстве было более наглядным.

Также приведем пример винтового движения объекта в пространстве вокруг произвольной оси. Ось будем задавать в сферических координатах:

```

1 function spherical_to_cartesian(θ_deg::Float64, φ_deg::Float64)
2     θ = deg2rad(θ_deg)
3     φ = deg2rad(φ_deg)
4     normalize([sin(θ)[club172098281|φ], sin(θ)φ, cos(θ)])
5 end
6
7 polar_angle = 60.0    # от оси Z
8 azimuth_angle = 45.0  # от оси X
9 axis_direction = spherical_to_cartesian(polar_angle,
    ↪ azimuth_angle)

```

Запустив программу, получим результат винтового движения объекта в пространстве вокруг произвольной оси, как показано на графике 4.2.

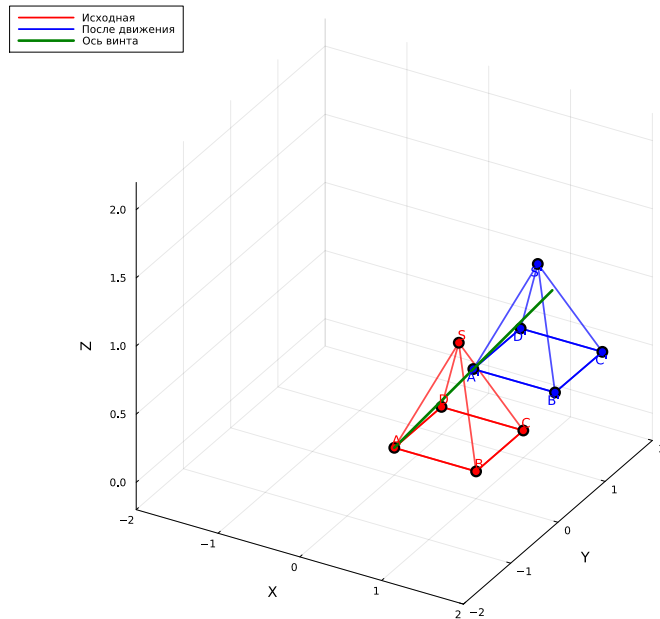


Рис. 4.2.: Винтовое движение пирамиды с параметрами вращения 180° и длиной трансляции равной единице вокруг оси Oz вокруг произвольной оси

4.5.2. Примеры отражения объекта

Попробуем теперь программно реализовать отражение пирамиды относительно плоскости. Для этого в блок с операциями над бикватернионами добавим некоторые формулы из раздела 4.4. Функция для задания плоскости будет выглядеть следующим образом:

```

1 function dq_plane(n::AbstractVector, d::Real)
2 @assert length(n) == 3 "Normal vector must have length 3"
3 return DualQuaternion(
4     Quaternion(0.0, n[1], n[2], n[3]),
5     Quaternion(-d, 0.0, 0.0, 0.0)
6 )
7 end

```

А для самой функции отражения точки будет верно:

```

1 function reflect_point(P::DualQuaternion, N::DualQuaternion)
2     return -N * complexconj(P) * N
3 end

```

Также можно добавить 4 ползунка для самой плоскости, 3 из которых регулируют вектор нормали и один для расстояния от начала координат:

```

1 @bind nx Slider(-1:0.01:1, default=0.0, show_value=true)
2 @bind ny Slider(-1:0.01:1, default=0.0, show_value=true)

```

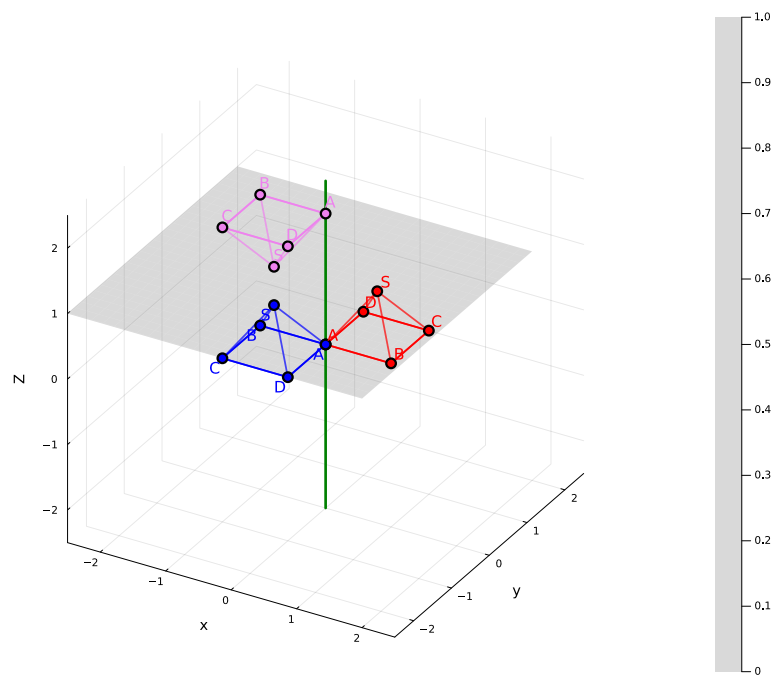


Рис. 4.3.: Винтовое движение пирамиды с параметрами вращения 180° и отражение ее относительно плоскости

```

3 @bind nz Slider(-1:0.01:1, default=1.0, show_value=true)
4 @bind d_plane Slider(-2:0.1:2, default=0.5, show_value=true)

```

Результат программы представлен на графике

5. Приложения

5.1. Приложение №1

```
1 begin
2     using Quaternions
3     using DualNumbers
4     using LinearAlgebra
5
6     # Вспомогательные функции для кватернионов
7     function dot(q1::Quaternion, q2::Quaternion)
8         return q1.s * q2.s + q1.v1 * q2.v1 + q1.v2 * q2.v2 +
↪ q1.v3 * q2.v3
9     end
10
11     function cross(q1::Quaternion, q2::Quaternion)
12         # Векторное произведение для чисто векторных кватернионов
13         return Quaternion(0.0,
14             q1.v2 * q2.v3 - q1.v3 * q2.v2,
15             q1.v3 * q2.v1 - q1.v1 * q2.v3,
16             q1.v1 * q2.v2 - q1.v2 * q2.v1
17         )
18     end
19
20     function conj(q::Quaternion)
21         return Quaternion(q.s, -q.v1, -q.v2, -q.v3)
22     end
23
24     function norm(q::Quaternion)
25         return sqrt(dot(q, q))
26     end
27
28     struct DualQuaternion
29         q::Quaternion{Float64} # вещественная часть
30         qo::Quaternion{Float64} # дуальная часть
31
32         # Конструктор из 4 дуальных чисел
33         function DualQuaternion(Q0::Dual, Q1::Dual, Q2::Dual,
↪ Q3::Dual)
34             q = Quaternion(Q0.value, Q1.value, Q2.value, Q3.value)
35             qo = Quaternion(Q0.epsilon, Q1.epsilon, Q2.epsilon,
↪ Q3.epsilon)
```

```

36         new(q, qo)
37     end
38
39     # Конструктор из двух кватернионов
40     DualQuaternion(q::Quaternion, qo::Quaternion) = new(q, qo)
41 end
42
43 # Арифметические операции
44 Base.:+(Q::DualQuaternion, P::DualQuaternion) =
↳ DualQuaternion(Q.q + P.q, Q.qo + P.qo)
45 Base.:-(Q::DualQuaternion) = DualQuaternion(-Q.q, -Q.qo)
46 Base.:-(Q::DualQuaternion, P::DualQuaternion) =
↳ DualQuaternion(Q.q - P.q, Q.qo - P.qo)
47
48 # Сложение с дуальным числом (только скалярная часть)
49 Base.:+(Q::DualQuaternion, d::Dual) = DualQuaternion(
50     Quaternion(Q.q.s + d.value, Q.q.v1, Q.q.v2, Q.q.v3),
51     Quaternion(Q.qo.s + d.epsilon, Q.qo.v1, Q.qo.v2, Q.qo.v3)
52 )
53 Base.:+(d::Dual, Q::DualQuaternion) = Q + d
54 Base.:-(Q::DualQuaternion, d::Dual) = Q + (-d)
55 Base.:-(d::Dual, Q::DualQuaternion) = DualQuaternion(
56     Quaternion(d.value - Q.q.s, -Q.q.v1, -Q.q.v2, -Q.q.v3),
57     Quaternion(d.epsilon - Q.qo.s, -Q.qo.v1, -Q.qo.v2,
↳ -Q.qo.v3)
58 )
59
60 # Умножение на число
61 Base.:*(Q::DualQuaternion, a::Real) = DualQuaternion(a * Q.q,
↳ a * Q.qo)
62 Base.:*(a::Real, Q::DualQuaternion) = Q * a
63
64 # Умножение на дуальное число
65 Base.:*(Q::DualQuaternion, d::Dual) = DualQuaternion(
66     d.value * Q.q,
67     d.value * Q.qo + d.epsilon * Q.q
68 )
69 Base.:*(d::Dual, Q::DualQuaternion) = Q * d
70
71 # Умножение двух дуальных кватернионов
72 Base.:*(Q::DualQuaternion, P::DualQuaternion) =
↳ DualQuaternion(
73     Q.q * P.q,
74     Q.q * P.qo + Q.qo * P.q
75 )
76
77 # Деление

```

```

78 Base.://(Q::DualQuaternion, a::Real) = DualQuaternion(Q.q / a,
↪ Q.qo / a)
79 Base.://(Q::DualQuaternion, d::Dual) = Q * inv(d)
80
81 # Сравнение
82 Base.:(==)(Q1::DualQuaternion, Q2::DualQuaternion) = (Q1.q ==
↪ Q2.q) && (Q1.qo == Q2.qo)
83
84 function Base.isapprox(Q1::DualQuaternion, Q2::DualQuaternion;
85 atol::Real=1e-12, rtol::Real=0)
86     return isapprox(Q1.q.s, Q2.q.s; atol, rtol) &&
87         isapprox(Q1.q.v1, Q2.q.v1; atol, rtol) &&
88         isapprox(Q1.q.v2, Q2.q.v2; atol, rtol) &&
89         isapprox(Q1.q.v3, Q2.q.v3; atol, rtol) &&
90         isapprox(Q1.qo.s, Q2.qo.s; atol, rtol) &&
91         isapprox(Q1.qo.v1, Q2.qo.v1; atol, rtol) &&
92         isapprox(Q1.qo.v2, Q2.qo.v2; atol, rtol) &&
93         isapprox(Q1.qo.v3, Q2.qo.v3; atol, rtol)
94 end
95
96 # Сопряжения
97 dualconj(Q::DualQuaternion) = DualQuaternion(Q.q, -Q.qo)
98 complexconj(Q::DualQuaternion) = DualQuaternion(conj(Q.q),
↪ conj(Q.qo))
99 conj(Q::DualQuaternion) = DualQuaternion(conj(Q.q),
↪ -conj(Q.qo))
100
101 # Нормы и инварианты
102 abs2(Q::DualQuaternion) = Dual(dot(Q.q, Q.q), 2 * dot(Q.q,
↪ Q.qo))
103 invariant(Q::DualQuaternion) = dot(Q.q, Q.qo)
104 parameter(Q::DualQuaternion) = invariant(Q) / dot(Q.q, Q.q)
105 abs(Q::DualQuaternion) = Dual(norm(Q.q), invariant(Q) /
↪ norm(Q.q))
106
107 # Конструкторы специальных объектов
108 function screw(v::AbstractVector, m::AbstractVector)
109     @assert length(v) == 3 && length(m) == 3 "Vectors must
↪ have length 3"
110     return DualQuaternion(
111         Quaternion(0.0, v[1], v[2], v[3]),
112         Quaternion(0.0, m[1], m[2], m[3])
113     )
114 end
115
116 function mass_point(w::Real, p::AbstractVector)
117     @assert length(p) == 3 "Point must have length 3"

```

```

118         return DualQuaternion(
119             Quaternion(w, 0.0, 0.0, 0.0),
120             Quaternion(0.0, p[1], p[2], p[3])
121         )
122     end
123
124     function dq_plane(n::AbstractVector, d::Real)
125         @assert length(n) == 3 "Normal vector must have length 3"
126         return DualQuaternion(
127             Quaternion(0.0, n[1], n[2], n[3]),
128             Quaternion(d, 0.0, 0.0, 0.0)
129         )
130     end
131
132     # Скалярное и векторное произведение
133     dot(P::DualQuaternion, Q::DualQuaternion) = Dual(
134         dot(P.q, Q.q),
135         dot(P.qo, Q.q) + dot(P.q, Q.qo)
136     )
137
138     function cross(P::DualQuaternion, Q::DualQuaternion)
139         @assert isapprox(P.q.s, 0.0) && isapprox(P.qo.s, 0.0) &&
140             isapprox(Q.q.s, 0.0) && isapprox(Q.qo.s, 0.0)
141         ↪ "Screw product requires pure dual quaternions"
142         return DualQuaternion(
143             cross(P.q, Q.q),
144             cross(P.q, Q.qo) + cross(P.qo, Q.q)
145         )
146     end
147
148     # Формулы для винтового движения
149     function RodriguesHamilton(A::DualQuaternion, θ::Dual)
150         Λθ = cos(θ/2)
151         Λ = sin(θ/2) * A
152         ↪ return L -> L + 2Λθ * cross(Λ, L) + 2 * cross(Λ, cross(Λ,
153             L))
154     end
155
156     function LineSandwichFormula(A::DualQuaternion, θ::Dual)
157         Λ = cos(θ/2) + sin(θ/2) * A
158         return L -> Λ * L * complexconj(Λ)
159     end
160
161     function PointSandwichFormula(A::DualQuaternion, θ::Dual)
162         Λ = cos(θ/2) + sin(θ/2) * A
163         return P -> Λ * P * conj(Λ)
164     end
165 end

```

```

163
164 PlaneSandwichFormula = PointSandwichFormula
165
166 # Вспомогательная функция для ортогональных векторов
167 function orthogonal_triple(n::AbstractVector)
168     @assert length(n) == 3 "Vector must have length 3"
169     if n[1] <= n[2] && n[1] <= n[3]
170         w = n + [1.0, 0.0, 0.0]
171     elseif n[2] <= n[1] && n[2] <= n[3]
172         w = n + [0.0, 1.0, 0.0]
173     else
174         w = n + [0.0, 0.0, 1.0]
175     end
176     u = normalize(cross(w, n))
177     v = normalize(cross(n, u))
178     return (v, u)
179 end
180
181
182 end
183
184 begin
185 points = [
186     [0.0, 0.0, 0.0],      # A
187     [1.0, 0.0, 0.0],      # B
188     [1.0, 1.0, 0.0],      # C
189     [0.0, 1.0, 0.0],      # D
190     [0.5, 0.5,  $\sqrt{2} / 2$ ] # S (вершина)
191 ]
192
193 function point_to_dualquaternion(p::Vector)
194     return DualQuaternion(
195         Quaternion(1.0, 0.0, 0.0, 0.0),
196         Quaternion(0.0, p[1], p[2], p[3])
197     )
198 end
199
200 function dualquaternion_to_point(dq::DualQuaternion)
201     return [dq.qo.v1, dq.qo.v2, dq.qo.v3]
202 end
203
204 # Грани пирамиды
205 base_faces = [[1,2,3,4]]
206 side_faces = [
207     [1,2,5], [2,3,5], [3,4,5], [4,1,5]
208 ]
209 end

```

```

210
211 function plot_pyramids(original, transformed)
212     p = plot(size=(1000, 800), legend=:topleft,
213             xlabel="X", ylabel="Y", zlabel="Z",
214             camera=(30, 30),
215             xlims=(-2, 2), ylims=(-2, 2), zlims=(-0.2, 2.2))
216
217     orig_x = [pt[1] for pt in original]
218     orig_y = [pt[2] for pt in original]
219     orig_z = [pt[3] for pt in original]
220
221     trans_x = [pt[1] for pt in transformed]
222     trans_y = [pt[2] for pt in transformed]
223     trans_z = [pt[3] for pt in transformed]
224
225     # Исходная пирамида
226     base_x = [orig_x[1], orig_x[2], orig_x[3], orig_x[4],
227             ↪ orig_x[1]]
228     base_y = [orig_y[1], orig_y[2], orig_y[3], orig_y[4],
229             ↪ orig_y[1]]
230     base_z = [orig_z[1], orig_z[2], orig_z[3], orig_z[4],
231             ↪ orig_z[1]]
232     plot!(p, base_x, base_y, base_z, color=:red, linewidth=2,
233             ↪ label="Исходная пирамида")
234
235     for face in side_faces
236         fx = [orig_x[i] for i in face]
237         fy = [orig_y[i] for i in face]
238         fz = [orig_z[i] for i in face]
239         plot!(p, fx, fy, fz, color=:red, linewidth=2, alpha=0.7,
240             ↪ label="")
241     end
242
243     # Преобразованная пирамида
244     trans_base_x = [trans_x[1], trans_x[2], trans_x[3],
245             ↪ trans_x[4], trans_x[1]]
246     trans_base_y = [trans_y[1], trans_y[2], trans_y[3],
247             ↪ trans_y[4], trans_y[1]]
248     trans_base_z = [trans_z[1], trans_z[2], trans_z[3],
249             ↪ trans_z[4], trans_z[1]]
250     plot!(p, trans_base_x, trans_base_y, trans_base_z,
251             ↪ color=:blue, linewidth=2, label="После винтового
252             ↪ движения")
253
254     for face in side_faces
255         fx = [trans_x[i] for i in face]
256         fy = [trans_y[i] for i in face]

```

```

247     fz = [trans_z[i] for i in face]
248     plot!(p, fx, fy, fz, color=:blue, linewidth=2, alpha=0.7,
           ↪ label="")
249 end
250
251 # Точки
252 scatter!(p, orig_x, orig_y, orig_z, color=:red, markersize=6,
           ↪ marker=:circle, label="")
253 scatter!(p, trans_x, trans_y, trans_z, color=:blue,
           ↪ markersize=6, marker=:circle, label="")
254
255 # Подписи
256 labels = ["A", "B", "C", "D", "S"]
257 offset = 0.05
258 for i in 1:length(original)
259     annotate!(p, orig_x[i] + offset, orig_y[i] + offset,
               ↪ orig_z[i] + offset,
260               text("$(labels[i])", :red, 10, :right,
261                 ↪ strokecolor=:white, strokewidth=1,
262                 ↪ backgroundcolor=RGBA(1,1,1,0.7)))
263     annotate!(p, trans_x[i] - offset, trans_y[i] - offset,
               ↪ trans_z[i] - offset,
264               text("$(labels[i])'", :blue, 10, :left,
265                 ↪ strokecolor=:white, strokewidth=1,
266                 ↪ backgroundcolor=RGBA(1,1,1,0.7)))
267 end
268
269 # Ось винта
270 plot!(p, [0, 0], [0, 0], [0, 2], color=:green, linewidth=3,
       ↪ arrow=true, label="Ось (Oz)")
271
272 return p
273 end
274
275 begin
276     angle = 2π
277     translation_distance = 1.0
278
279     angle_slider = @bind angle Slider(0:0.01:2π, default=π,
       ↪ show_value=true)
280     trans_slider = @bind translation_distance Slider(0:0.01:1.0,
       ↪ default=1.0, show_value=true)
281
282 md"""
283 ### Управление винтовым движением
284 Угол поворота (рад): $(angle_slider)

```

```

284 Трансляция вдоль оси: $(trans_slider)
285 """
286 End
287
288 begin
289   # Ось винта – фиксирована (Oz)
290   axis_direction = normalize([0.0, 0.0, 1.0])
291   A = DualQuaternion(
292     Quaternion(0.0, axis_direction...),
293     Quaternion(0.0, 0.0, 0.0, 0.0)
294   )
295
296   θ = Dual(angle, translation_distance)
297   motor_func = PointSandwichFormula(A, θ)
298
299   original_points = copy(points)
300   transformed_points = []
301
302   for p in points
303     point_dq = point_to_dualquaternion(p)
304     transformed_dq = motor_func(point_dq)
305     transformed_p = dualquaternion_to_point(transformed_dq)
306     push!(transformed_points, transformed_p)
307   end
308
309   plot_pyramids(original_points, transformed_points)
310 end
311

```

5.2. Приложение №2

```

1  begin
2  using Quaternions
3  using DualNumbers
4  using LinearAlgebra
5
6  # Вспомогательные функции для кватернионов
7  function dot(q1::Quaternion, q2::Quaternion)
8      return q1.s * q2.s + q1.v1 * q2.v1 + q1.v2 * q2.v2 + q1.v3 *
          ↪ q2.v3
9  end
10
11 function cross(q1::Quaternion, q2::Quaternion)
12     # Векторное произведение для чисто векторных кватернионов
13     return Quaternion(0.0,
14         q1.v2 * q2.v3 - q1.v3 * q2.v2,

```

```

15         q1.v3 * q2.v1 - q1.v1 * q2.v3,
16         q1.v1 * q2.v2 - q1.v2 * q2.v1
17     )
18 end
19
20 function conj(q::Quaternion)
21     return Quaternion(q.s, -q.v1, -q.v2, -q.v3)
22 end
23
24 function norm(q::Quaternion)
25     return sqrt(dot(q, q))
26 end
27
28 struct DualQuaternion
29     q::Quaternion{Float64} # вещественная часть
30     qo::Quaternion{Float64} # дуальная часть
31
32     # Конструктор из 4 дуальных чисел
33     function DualQuaternion(Q0::Dual, Q1::Dual, Q2::Dual,
34         ↪ Q3::Dual)
35         q = Quaternion(Q0.value, Q1.value, Q2.value, Q3.value)
36         qo = Quaternion(Q0.epsilon, Q1.epsilon, Q2.epsilon,
37             ↪ Q3.epsilon)
38         new(q, qo)
39     end
40
41     # Конструктор из двух кватернионов
42     DualQuaternion(q::Quaternion, qo::Quaternion) = new(q, qo)
43 end
44
45 # Арифметические операции
46 Base.:+(Q::DualQuaternion, P::DualQuaternion) =
47     ↪ DualQuaternion(Q.q + P.q, Q.qo + P.qo)
48 Base.:-(Q::DualQuaternion) = DualQuaternion(-Q.q, -Q.qo)
49 Base.:-(Q::DualQuaternion, P::DualQuaternion) =
50     ↪ DualQuaternion(Q.q - P.q, Q.qo - P.qo)
51
52 # Сложение с дуальным числом (только скалярная часть)
53 Base.:+(Q::DualQuaternion, d::Dual) = DualQuaternion(
54     Quaternion(Q.q.s + d.value, Q.q.v1, Q.q.v2, Q.q.v3),
55     Quaternion(Q.qo.s + d.epsilon, Q.qo.v1, Q.qo.v2, Q.qo.v3)
56 )
57 Base.:+(d::Dual, Q::DualQuaternion) = Q + d
58 Base.:-(Q::DualQuaternion, d::Dual) = Q + (-d)
59 Base.:-(d::Dual, Q::DualQuaternion) = DualQuaternion(
60     Quaternion(d.value - Q.q.s, -Q.q.v1, -Q.q.v2, -Q.q.v3),
61     Quaternion(d.epsilon - Q.qo.s, -Q.qo.v1, -Q.qo.v2, -Q.qo.v3)

```

```

58 )
59
60 # Умножение на число
61 Base.:(Q::DualQuaternion, a::Real) = DualQuaternion(a * Q.q, a *
    ↪ Q.qo)
62 Base.:(a::Real, Q::DualQuaternion) = Q * a
63
64 # Умножение на дуальное число
65 Base.:(Q::DualQuaternion, d::Dual) = DualQuaternion(
66     d.value * Q.q,
67     d.value * Q.qo + d.epsilon * Q.q
68 )
69 Base.:(d::Dual, Q::DualQuaternion) = Q * d
70
71 # Умножение двух дуальных кватернионов
72 Base.:(Q::DualQuaternion, P::DualQuaternion) = DualQuaternion(
73     Q.q * P.q,
74     Q.q * P.qo + Q.qo * P.q
75 )
76
77 # Деление
78 Base.:(Q::DualQuaternion, a::Real) = DualQuaternion(Q.q / a,
    ↪ Q.qo / a)
79 Base.:(Q::DualQuaternion, d::Dual) = Q * inv(d)
80
81 # Сравнение
82 Base.:(==)(Q1::DualQuaternion, Q2::DualQuaternion) = (Q1.q ==
    ↪ Q2.q) && (Q1.qo == Q2.qo)
83
84 function Base.isapprox(Q1::DualQuaternion, Q2::DualQuaternion;
85     atol::Real=1e-12, rtol::Real=0)
86     return isapprox(Q1.q.s, Q2.q.s; atol, rtol) &&
87         isapprox(Q1.q.v1, Q2.q.v1; atol, rtol) &&
88         isapprox(Q1.q.v2, Q2.q.v2; atol, rtol) &&
89         isapprox(Q1.q.v3, Q2.q.v3; atol, rtol) &&
90         isapprox(Q1.qo.s, Q2.qo.s; atol, rtol) &&
91         isapprox(Q1.qo.v1, Q2.qo.v1; atol, rtol) &&
92         isapprox(Q1.qo.v2, Q2.qo.v2; atol, rtol) &&
93         isapprox(Q1.qo.v3, Q2.qo.v3; atol, rtol)
94 end
95
96 # Сопряжения
97 dualconj(Q::DualQuaternion) = DualQuaternion(Q.q, -Q.qo)
98 complexconj(Q::DualQuaternion) = DualQuaternion(conj(Q.q),
    ↪ conj(Q.qo))
99 conj(Q::DualQuaternion) = DualQuaternion(conj(Q.q), -conj(Q.qo))
100

```

```

101 # Нормы и инварианты
102 abs2(Q::DualQuaternion) = Dual(dot(Q.q, Q.q), 2 * dot(Q.q, Q.qo))
103 invariant(Q::DualQuaternion) = dot(Q.q, Q.qo)
104 parameter(Q::DualQuaternion) = invariant(Q) / dot(Q.q, Q.q)
105 abs(Q::DualQuaternion) = Dual(norm(Q.q), invariant(Q) / norm(Q.q))
106
107 # Конструкторы специальных объектов
108 function screw(v::AbstractVector, m::AbstractVector)
109     @id141972837 (@assert) length(v) == 3 && length(m) == 3
110     ↪ "Vectors must have length 3"
111     return DualQuaternion(
112         Quaternion(0.0, v[1], v[2], v[3]),
113         Quaternion(0.0, m[1], m[2], m[3])
114     )
115 end
116
117 function mass_point(w::Real, p::AbstractVector)
118     @id141972837 (@assert) length(p) ==
119     3 "Point must have length 3"
120     return DualQuaternion(
121         Quaternion(w, 0.0, 0.0, 0.0),
122         Quaternion(0.0, p[1], p[2], p[3])
123     )
124 end
125
126 function dq_plane(n::AbstractVector, d::Real)
127     @id141972837 (@assert) length(n) == 3 "Normal vector must have
128     ↪ length 3"
129     return DualQuaternion(
130         Quaternion(0.0, n[1], n[2], n[3]),
131         Quaternion(d, 0.0, 0.0, 0.0)
132     )
133 end
134
135 # Скалярное и векторное произведение
136 dot(P::DualQuaternion, Q::DualQuaternion) = Dual(
137     dot(P.q, Q.q),
138     dot(P.qo, Q.q) + dot(P.q, Q.qo)
139 )
140
141 function cross(P::DualQuaternion, Q::DualQuaternion)
142     @id141972837 (@assert) isapprox(P.q.s, 0.0) &&
143     ↪ isapprox(P.qo.s, 0.0) &&
144     isapprox(Q.q.s, 0.0) && isapprox(Q.qo.s, 0.0) "Screw
145     ↪ product requires pure dual quaternions"
146     return DualQuaternion(
147         cross(P.q, Q.q),

```

```

144         cross(P.q, Q.qo) + cross(P.qo, Q.q)
145     )
146 end
147
148 # Формулы для винтового движения
149 function RodriguesHamilton(A::DualQuaternion, θ::Dual)
150     Λ0 = cos(θ/2)
151     Λ = sin(θ/2) * A
152     return L -> L + 2Λ0 * cross(Λ, L) + 2 * cross(Λ, cross(Λ, L))
153 end
154
155 function LineSandwichFormula(A::DualQuaternion, θ::Dual)
156     Λ = cos(θ/2) + sin(θ/2) * A
157     return L -> Λ * L * complexconj(Λ)
158 end
159
160 function PointSandwichFormula(A::DualQuaternion, θ::Dual)
161     Λ = cos(θ/2) + sin(θ/2) * A
162     return P -> Λ * P * conj(Λ)
163 end
164
165 PlaneSandwichFormula = PointSandwichFormula
166
167 # Вспомогательная функция для ортогональных векторов
168 function orthogonal_triple(n::AbstractVector)
169     @id141972837 (@assert) length(n) == 3 "Vector must have length
170         ↪ 3"
171     if n[1] <= n[2] && n[1] <= n[3]
172         w = n + [1.0, 0.0, 0.0]
173     elseif n[2] <= n[1] && n[2] <= n[3]
174         w = n + [0.0, 1.0, 0.0]
175     else
176         w = n + [0.0, 0.0, 1.0]
177     end
178     u = normalize(cross(w, n))
179     v = normalize(cross(n, u))
180     return (v, u)
181 end
182
183 end
184
185 begin
186     using Plots
187     using Colors
188     using PlutoUI
189 end

```

```

190
191 begin
192
193     angle = 2π
194     translation_distance = 1.0
195
196     angle_slider = @bind angle Slider(0:0.01:2π, default=π,
197         ↪ show_value=true)
198     trans_slider = @bind translation_distance Slider(0:0.01:1.0,
199         ↪ default=1.0, show_value=true)
200
201     md"""
202     ### Управление винтовым движением
203     Угол поворота (рад): $(angle_slider)
204     Трансляция вдоль оси: $(trans_slider)
205     """
206 end
207
208 begin
209     points = [
210         [0.0, 0.0, 0.0],      # A
211         [1.0, 0.0, 0.0],      # B
212         [1.0, 1.0, 0.0],      # C
213         [0.0, 1.0, 0.0],      # D
214         [0.5, 0.5, √2 / 2]     # S
215     ]
216
217     base_faces = [[1,2,3,4]]
218     side_faces = [[1,2,5], [2,3,5], [3,4,5], [4,1,5]]
219
220     б function point_to_dualquaternion(p::Vector)
221         return DualQuaternion(
222             Quaternion(1.0, 0.0, 0.0, 0.0),
223             Quaternion(0.0, p[1], p[2], p[3])
224         )
225     end
226
227     function dualquaternion_to_point(dq::DualQuaternion)
228         return [dq.qo.v1, dq.qo.v2, dq.qo.v3]
229     end
230
231     function plot_pyramids_with_arbitrary_axis(original,
232         ↪ transformed, axis_vector)
233         p = plot(size=(1000, 800), legend=:topleft,
234             xlabel="X", ylabel="Y", zlabel="Z",
235             camera=(30, 30),
236             xlims=(-2, 2), ylims=(-2, 2), zlims=(-0.2, 2.2))

```

```

234 orig_x = [pt[1] for pt in original]
235 orig_y = [pt[2] for pt in original]
236 orig_z = [pt[3] for pt in original]
237
238
239 trans_x = [pt[1] for pt in transformed]
240 trans_y = [pt[2] for pt in transformed]
241 trans_z = [pt[3] for pt in transformed]
242
243 base_x = [orig_x[1], orig_x[2], orig_x[3], orig_x[4],
    ↪ orig_x[1]]
244 base_y = [orig_y[1], orig_y[2], orig_y[3], orig_y[4],
    ↪ orig_y[1]]
245 base_z = [orig_z[1], orig_z[2], orig_z[3], orig_z[4],
    ↪ orig_z[1]]
246 plot!(p, base_x, base_y, base_z, color=:red, linewidth=2,
    ↪ label="Исходная")
247
248 for face in side_faces
249     fx = [orig_x[i] for i in face]
250     fy = [orig_y[i] for i in face]
251     fz = [orig_z[i] for i in face]
252     plot!(p, fx,
253     fy, fz, color=:red, linewidth=2, alpha=0.7, label="")
254 end
255
256 tbase_x = [trans_x[1], trans_x[2], trans_x[3],
    ↪ trans_x[4], trans_x[1]]
257 tbase_y = [trans_y[1], trans_y[2], trans_y[3],
    ↪ trans_y[4], trans_y[1]]
258 tbase_z = [trans_z[1], trans_z[2], trans_z[3],
    ↪ trans_z[4], trans_z[1]]
259 plot!(p, tbase_x, tbase_y, tbase_z, color=:blue,
    ↪ linewidth=2, label="После движения")
260
261 for face in side_faces
262     fx = [trans_x[i] for i in face]
263     fy = [trans_y[i] for i in face]
264     fz = [trans_z[i] for i in face]
265     plot!(p, fx, fy, fz, color=:blue, linewidth=2,
    ↪ alpha=0.7, label="")
266 end
267
268 scatter!(p, orig_x, orig_y, orig_z, color=:red,
    ↪ markersize=6, label="")
269 scatter!(p, trans_x, trans_y, trans_z, color=:blue,
    ↪ markersize=6, label="")

```

```

270 labels = ["A", "B", "C", "D", "S"]
271 offset = 0.05
272
273 for i in 1:length(original)
274     annotate!(p, orig_x[i] + offset, orig_y[i] + offset,
275         ↪ orig_z[i] + offset,
276         text("$ (labels[i])", :red, 10, :right,
277             strokecolor=:white, strokewidth=2,
278             ↪ backgroundcolor=RGBA(1,1,1,0.7)))
279     annotate!(p, trans_x[i] - offset, trans_y[i] -
280         ↪ offset, trans_z[i] - offset,
281         text("$ (labels[i])'", :blue, 10, :left,
282             strokecolor=:white, strokewidth=2,
283             ↪ backgroundcolor=RGBA(1,1,1,0.7)))
284
285 end
286
287 # Ось винта
288 axis_end = axis_vector .* 2.0
289 plot!(p, [0, axis_end[1]], [0, axis_end[2]], [0,
290     ↪ axis_end[3]],
291     color=:green, linewidth=3, arrow=true, label="Ось
292     ↪ винта")
293
294 return p
295 end
296
297 function spherical_to_cartesian(θ_deg::Float64,
298     ↪ φ_deg::Float64)
299     θ = deg2rad(θ_deg)
300     φ = deg2rad(φ_deg)
301     normalize([sin(θ)[club172098281|φ], sin(θ)φ, cos(θ)])
302 end
303
304 polar_angle = 60.0 # от оси Z
305 azimuth_angle = 45.0 # от оси X
306 axis_direction = spherical_to_cartesian(polar_angle,
307     ↪ azimuth_angle)
308
309 A = DualQuaternion(
310     Quaternion(0.0, axis_direction...),
311     Quaternion(0.0, 0.0, 0.0, 0.0)
312 )
313 θ_dual = Dual(angle, translation_distance)
314 motor_func = PointSandwichFormula(A, θ_dual)
315
316 # === Преобразование точек ===
317 original_points = copy(points)

```

```

309     transformed_points = [
310         dualquaternion_to_point(motor_func(point_to_dualquaternionio,
311             ↪ n(p)))
312     ]
313     savefig("2.pdf")
314     plot_pyramids_with_arbitrary_axis(original_points,
315         ↪ transformed_points, axis_direction)
316
317 end

```

5.3. Приложение №3

```

1  begin
2      using Quaternions
3      using DualNumbers
4      using LinearAlgebra
5
6      # Вспомогательные функции для кватернионов
7      function dot(q1::Quaternion, q2::Quaternion)
8          return q1.s * q2.s + q1.v1 * q2.v1 + q1.v2 * q2.v2 +
9             ↪ q1.v3 * q2.v3
10         end
11
12     function cross(q1::Quaternion, q2::Quaternion)
13         # Векторное произведение для чисто векторных кватернионов
14         return Quaternion(0.0,
15             q1.v2 * q2.v3 - q1.v3 * q2.v2,
16             q1.v3 * q2.v1 - q1.v1 * q2.v3,
17             q1.v1 * q2.v2 - q1.v2 * q2.v1
18         )
19     end
20
21     function conj(q::Quaternion)
22         return Quaternion(q.s, -q.v1, -q.v2, -q.v3)
23     end
24
25     function norm(q::Quaternion)
26         return sqrt(dot(q, q))
27     end
28
29     struct DualQuaternion
30         q::Quaternion{Float64} # вещественная часть
31         qo::Quaternion{Float64} # дуальная часть

```

```

32     # Конструктор из 4 дуальных чисел
33     function DualQuaternion(Q0::Dual, Q1::Dual, Q2::Dual,
↪ Q3::Dual)
34         q = Quaternion(Q0.value, Q1.value, Q2.value, Q3.value)
35         qo = Quaternion(Q0.epsilon, Q1.epsilon, Q2.epsilon,
↪ Q3.epsilon)
36         new(q, qo)
37     end
38
39     # Конструктор из двух кватернионов
40     DualQuaternion(q::Quaternion, qo::Quaternion) = new(q, qo)
41 end
42
43 # Арифметические операции
44 Base.:+(Q::DualQuaternion, P::DualQuaternion) =
↪ DualQuaternion(Q.q + P.q, Q.qo + P.qo)
45 Base.:-(Q::DualQuaternion) = DualQuaternion(-Q.q, -Q.qo)
46 Base.:-(Q::DualQuaternion, P::DualQuaternion) =
↪ DualQuaternion(Q.q - P.q, Q.qo - P.qo)
47
48 # Сложение с дуальным числом (только скалярная часть)
49 Base.:+(Q::DualQuaternion, d::Dual) = DualQuaternion(
50     Quaternion(Q.q.s + d.value, Q.q.v1, Q.q.v2, Q.q.v3),
51     Quaternion(Q.qo.s + d.epsilon, Q.qo.v1, Q.qo.v2, Q.qo.v3)
52 )
53 Base.:+(d::Dual, Q::DualQuaternion) = Q + d
54 Base.:-(Q::DualQuaternion, d::Dual) = Q + (-d)
55 Base.:-(d::Dual, Q::DualQuaternion) = DualQuaternion(
56     Quaternion(d.value - Q.q.s, -Q.q.v1, -Q.q.v2, -Q.q.v3),
57     Quaternion(d.epsilon - Q.qo.s, -Q.qo.v1, -Q.qo.v2,
↪ -Q.qo.v3)
58 )
59
60 # Умножение на число
61 Base.:*(Q::DualQuaternion, a::Real) = DualQuaternion(a * Q.q,
↪ a * Q.qo)
62 Base.:*(a::Real, Q::DualQuaternion) = Q * a
63
64 # Умножение на дуальное число
65 Base.:*(Q::DualQuaternion, d::Dual) = DualQuaternion(
66     d.value * Q.q,
67     d.value * Q.qo + d.epsilon * Q.q
68 )
69 Base.:*(d::Dual, Q::DualQuaternion) = Q * d
70
71 # Умножение двух дуальных кватернионов

```

```

72 Base.*(Q::DualQuaternion, P::DualQuaternion) =
↳ DualQuaternion(
73     Q.q * P.q,
74     Q.q * P.qo + Q.qo * P.q
75 )
76
77 # Деление
78 Base.:(Q::DualQuaternion, a::Real) = DualQuaternion(Q.q / a,
↳ Q.qo / a)
79 Base.:(Q::DualQuaternion, d::Dual) = Q * inv(d)
80
81 # Сравнение
82 Base.:(Q1::DualQuaternion, Q2::DualQuaternion) = (Q1.q ==
↳ Q2.q) && (Q1.qo == Q2.qo)
83
84 function Base.isapprox(Q1::DualQuaternion, Q2::DualQuaternion;
85     atol::Real=1e-12, rtol::Real=0)
86     return isapprox(Q1.q.s, Q2.q.s; atol, rtol) &&
87         isapprox(Q1.q.v1, Q2.q.v1; atol, rtol) &&
88         isapprox(Q1.q.v2, Q2.q.v2; atol, rtol) &&
89         isapprox(Q1.q.v3, Q2.q.v3; atol, rtol) &&
90         isapprox(Q1.qo.s, Q2.qo.s; atol, rtol) &&
91         isapprox(Q1.qo.v1, Q2.qo.v1; atol, rtol) &&
92         isapprox(Q1.qo.v2, Q2.qo.v2; atol, rtol) &&
93         isapprox(Q1.qo.v3, Q2.qo.v3; atol, rtol)
94 end
95
96 # Сопряжения
97 dualconj(Q::DualQuaternion) = DualQuaternion(Q.q, -Q.qo)
98 complexconj(Q::DualQuaternion) = DualQuaternion(conj(Q.q),
↳ conj(Q.qo))
99 conj(Q::DualQuaternion) = DualQuaternion(conj(Q.q),
↳ -conj(Q.qo))
100
101 # Нормы и инварианты
102 abs2(Q::DualQuaternion) = Dual(dot(Q.q, Q.q), 2 * dot(Q.q,
↳ Q.qo))
103 invariant(Q::DualQuaternion) = dot(Q.q, Q.qo)
104 parameter(Q::DualQuaternion) = invariant(Q) / dot(Q.q, Q.q)
105 abs(Q::DualQuaternion) = Dual(norm(Q.q), invariant(Q) /
↳ norm(Q.q))
106
107 # Конструкторы специальных объектов
108 function screw(v::AbstractVector, m::AbstractVector)
109     @assert length(v) == 3 && length(m) == 3 "Vectors must
↳ have length 3"
110     return DualQuaternion(

```

```

111         Quaternion(0.0, v[1], v[2], v[3]),
112         Quaternion(0.0, m[1], m[2], m[3])
113     )
114 end
115
116 function mass_point(w::Real, p::AbstractVector)
117     @assert length(p) == 3 "Point must have length 3"
118     return DualQuaternion(
119         Quaternion(w, 0.0, 0.0, 0.0),
120         Quaternion(0.0, p[1], p[2], p[3])
121     )
122 end
123
124 function dq_plane(n::AbstractVector, d::Real)
125     @assert length(n) == 3 "Normal vector must have length 3"
126     return DualQuaternion(
127         Quaternion(0.0, n[1], n[2], n[3]),
128         Quaternion(d, 0.0, 0.0, 0.0)
129     )
130 end
131
132 # Скалярное и векторное произведение
133 dot(P::DualQuaternion, Q::DualQuaternion) = Dual(
134     dot(P.q, Q.q),
135     dot(P.qo, Q.q) + dot(P.q, Q.qo)
136 )
137
138 function cross(P::DualQuaternion, Q::DualQuaternion)
139     @assert isapprox(P.q.s, 0.0) && isapprox(P.qo.s, 0.0) &&
140         isapprox(Q.q.s, 0.0) && isapprox(Q.qo.s, 0.0)
141     ↪ "Screw product requires pure dual quaternions"
142     return DualQuaternion(
143         cross(P.q, Q.q),
144         cross(P.q, Q.qo) + cross(P.qo, Q.q)
145     )
146 end
147
148 # Формулы для винтового движения
149 function RodriguesHamilton(A::DualQuaternion, θ::Dual)
150     Λ0 = cos(θ/2)
151     Λ = sin(θ/2) * A
152     ↪ return L -> L + 2Λ0 * cross(Λ, L) + 2 * cross(Λ, cross(Λ,
153     L))
154 end
155
156 function LineSandwichFormula(A::DualQuaternion, θ::Dual)
157     Λ = cos(θ/2) + sin(θ/2) * A

```

```

156         return L -> Λ * L * complexconj(Λ)
157     end
158
159     function PointSandwichFormula(A::DualQuaternion, Θ::Dual)
160         Λ = cos(Θ/2) + sin(Θ/2) * A
161         return P -> Λ * P * conj(Λ)
162     end
163
164     PlaneSandwichFormula = PointSandwichFormula
165
166     # Вспомогательная функция для ортогональных векторов
167     function orthogonal_triple(n::AbstractVector)
168         @assert length(n) == 3 "Vector must have length 3"
169         if n[1] <= n[2] && n[1] <= n[3]
170             w = n + [1.0, 0.0, 0.0]
171         elseif n[2] <= n[1] && n[2] <= n[3]
172             w = n + [0.0, 1.0, 0.0]
173         else
174             w = n + [0.0, 0.0, 1.0]
175         end
176         u = normalize(cross(w, n))
177         v = normalize(cross(n, u))
178         return (v, u)
179     end
180
181
182     end
183     begin
184         # Определение точки P (аффинная точка)
185         point_coords = [0.5, 0.5, √2 / 2]
186         m = DualQuaternion(Quaternion(1.0, 0.0, 0.0, 0.0),
187             ↪ Quaternion(0.0, point_coords...)) # m=1+(xi+yj+zk)ε
188     end
189
190     begin
191         # Разделы 5.4 и 5.3
192         # Определение оси вращения (проходящую через начало координат)
193         # Например, ось Z: s = (0, 0, 1)
194         axis_direction = normalize([0.0, 0.0, 1.0]) # Проверить, что
195             ↪ нормирован
196         # Момент для оси через O: s_0 = (0, 0, 0)
197         A = DualQuaternion(Quaternion(0.0, axis_direction...),
198             ↪ Quaternion(0.0, 0.0, 0.0, 0.0))
199         # Должно удовлетворяться условие Плюккера
200     end
201
202     begin

```

```

200 # На 90 градусов
201 angle =  $\pi/2$ 
202  $\Theta$  = Dual(angle, 0.0)
203 end
204
205 # Бикватернион винтового движения определен в блоке выше (формула
    ↪ 12, стр 19)
206 # Предполагается использование сэндвич-формулы (раздел 5.4.1),
    ↪ бикватернионное сопряжение раздел 5.2.5
207 rot_func = PointSandwichFormula(A,  $\Theta$ )
208
209 begin
210 # Применяем вращение
211 m_rotated = rot_func(m)
212
213 println("Исходное представление точки: ", m)
214 println("Представление после вращения: ", m_rotated)
215 end
216
217 begin
218 # Извлекаем координаты вращенной точки из m_rotated.qo (векторная
    ↪ часть дуальной части)
219 # потому что m_rotated должно быть вида  $(1, 0, 0, 0) + (0, x', y',$ 
    ↪  $z')*\epsilon$ 
220 # для аффинной точки  $(x', y', z', 1)$ 
221 rotated_point_coords = [m_rotated.qo.v1, m_rotated.qo.v2,
    ↪ m_rotated.qo.v3]
222 println("Исходные координаты точки: ", point_coords)
223 println("Координаты точки после вращения: ", rotated_point_coords)
224
225 end
226
227 begin
228 using Plots
229 using Colors
230 end
231
232 points = [
233     [0.0, 0.0, 0.0],      # A
234     [1.0, 0.0, 0.0],      # B
235     [1.0, 1.0, 0.0],      # C
236     [0.0, 1.0, 0.0],      # D
237     [0.5, 0.5,  $\sqrt{2} / 2$ ] # S (вершина)
238 ]
239
240 # Функция для преобразования точки в бикватернион
241 function point_to_dualquaternion(p::Vector)

```

```

242     return DualQuaternion(
243         Quaternion(1.0, 0.0, 0.0, 0.0),
244         Quaternion(0.0, p[1], p[2], p[3])
245     )
246 end
247
248 # Функция для извлечения координат из бикватерниона точки
249 function dualquaternion_to_point(dq::DualQuaternion)
250     return [dq.qo.v1, dq.qo.v2, dq.qo.v3]
251 end
252
253 begin
254     # Параметры винтового движения
255     angle1 =  $\pi$ 
256     translation_distance = 0.0
257      $\theta_1$  = Dual(angle1, translation_distance)
258 end
259
260 begin
261     # Ось винтового движения (Oz)
262     axis_direction1 = normalize([0.0, 0.0, 1.0])
263     A1 = DualQuaternion(
264         Quaternion(0.0, axis_direction1...),
265         Quaternion(0.0, 0.0, 0.0, 0.0)
266     )
267 end
268
269 motor_func = PointSandwichFormula(A1,  $\theta_1$ )
270
271 begin
272     # Применяем винтовое движение ко всем точкам
273     original_points = []
274     transformed_points = []
275
276     for p in points
277         # Преобразуем точку в бикватернион
278         point_dq = point_to_dualquaternion(p)
279
280         # Применяем винтовое движение
281         transformed_dq = motor_func(point_dq)
282
283         # Извлекаем координаты
284         transformed_p = dualquaternion_to_point(transformed_dq)
285
286         push!(original_points, p)
287         push!(transformed_points, transformed_p)
288     end

```

```

289 end
290
291 begin
292     # Основание ABCD
293     base_faces = [[1,2,3,4]] # A-B-C-D
294
295     # Боковые грани
296     side_faces = [
297         [1,2,5], # A-B-S
298         [2,3,5], # B-C-S
299         [3,4,5], # C-D-S
300         [4,1,5] # D-A-S
301     ]
302
303 end
304
305 begin
306     normal_vec = [nx, ny, nz]
307     n_len = LinearAlgebra.norm(normal_vec)
308
309     if n_len > 0
310         normal_vec = normal_vec / n_len
311     else
312         normal_vec = [0.0, 0.0, 1.0]
313     end
314
315     reflection_plane = dq_plane(normal_vec, d_plane)
316
317     reflected_points = []
318     for p in transformed_points
319         dq = point_to_dualquaternion(p)
320         # Используйте правильную формулу
321         dq_reflected = reflect_point(dq, reflection_plane)
322         p_reflected = dualquaternion_to_point(dq_reflected)
323         push!(reflected_points, p_reflected)
324     end
325
326 end
327
328 using Statistics
329
330 function plot_pyramids(original, transformed, reflected, normal,
    ↪ d)
331
332
333     p = plot(size=(1000, 800), legend=:topleft, xlabel="x",
    ↪ ylabel="y", zlabel="Z",

```

```

334         camera=(30, 30), xlims=(-2.5, 2.5), ylims=(-2.5,
335             ↪ 2.5), zlims=(-2.5, 2.5),
336         grid=True, aspect_ratio=equal)
337
338     center_x = mean([pt[1] for pt in transformed])
339     center_y = mean([pt[2] for pt in transformed])
340     center_z = mean([pt[3] for pt in transformed])
341     center = [center_x, center_y,
342         center_z]
343
344     n_dot_c = LinearAlgebra.dot(normal, center)
345     dist = n_dot_c - d
346     center_on_plane = center - dist .* normal
347
348     range_size = 2.5
349
350     xs = range(center_on_plane[1] - range_size,
351         ↪ center_on_plane[1] + range_size, length=30)
352     ys = range(center_on_plane[2] - range_size,
353         ↪ center_on_plane[2] + range_size, length=30)
354
355     if abs(normal[3]) > 1e-6
356         zs = [(d - normal[1]*x - normal[2]*y) / normal[3] for x
357             ↪ in xs, y in ys]
358         zs = clamp.(zs, -2.5, 2.5)
359         surface!(p, xs, ys, zs, color=:gray, alpha=0.3,
360             ↪ legend=false)
361     elseif abs(normal[2]) > 1e-6
362         # Если плоскость вертикальна вдоль Y
363         zs = range(center_on_plane[3] - range_size,
364             ↪ center_on_plane[3] + range_size, length=30)
365         ys_calc = [(d - normal[1]*x - normal[3]*z) / normal[2]
366             ↪ for x in xs, z in zs]
367         ys_calc = clamp.(ys_calc, -2.5, 2.5)
368         surface!(p, xs, ys_calc, zs, color=:gray, alpha=0.3,
369             ↪ legend=false)
370     else
371         # Если плоскость вертикальна вдоль X
372         zs = range(center_on_plane[3] - range_size,
373             ↪ center_on_plane[3] + range_size, length=30)
374         ys_calc = range(center_on_plane[2] - range_size,
375             ↪ center_on_plane[2] + range_size, length=30)
376         xs_calc = [(d - normal[2]*y - normal[3]*z) / normal[1]
377             ↪ for y in ys_calc, z in zs]
378         xs_calc = clamp.(xs_calc, -2.5, 2.5)
379         surface!(p, xs_calc, ys_calc, zs, color=:gray, alpha=0.3,
380             ↪ legend=false)

```

```

369 end
370
371 plot!(p, [0, 0], [0, 0], [-2.5, 2.5], color=:green,
    ↪ linewidth=3, arrow=true, label="Ось винта")
372
373 orig_x = [pt[1] for pt in original]
374 orig_y = [pt[2] for pt in original]
375 orig_z = [pt[3] for pt in original]
376
377 base_x = [orig_x[1], orig_x[2], orig_x[3], orig_x[4],
    ↪ orig_x[1]]
378 base_y = [orig_y[1], orig_y[2], orig_y[3], orig_y[4],
    ↪ orig_y[1]]
379 base_z = [orig_z[1], orig_z[2], orig_z[3], orig_z[4],
    ↪ orig_z[1]]
380
381 plot!(p, base_x, base_y, base_z, color=:red, linewidth=2,
    ↪ label="Исходная")
382
383 side_faces = [[1,2,5],[2,3,5],[3,4,5],[4,1,5]]
384 for face in side_faces
385     fx = [orig_x[i] for i in face]
386     fy = [orig_y[i] for i in face]
387     fz = [orig_z[i] for i in face]
388     plot!(p, fx, fy, fz, color=:red, linewidth=2, alpha=0.7,
    ↪ label="")
389 end
390 scatter!(p, orig_x, orig_y, orig_z, color=:red, markersize=6,
    ↪ label="")
391
392 trans_x = [pt[1] for pt in transformed]
393 trans_y = [pt[2] for pt in transformed]
394 trans_z = [pt[3] for pt in transformed]
395
396 trans_base_x = [trans_x[1], trans_x[2], trans_x[3],
    ↪ trans_x[4], trans_x[1]]
397 trans_base_y = [trans_y[1], trans_y[2], trans_y[3],
    ↪ trans_y[4], trans_y[1]]
398 trans_base_z = [trans_z[1], trans_z[2], trans_z[3],
    ↪ trans_z[4], trans_z[1]]
399
400 plot!(p, trans_base_x, trans_base_y, trans_base_z,
    ↪ color=:blue, linewidth=2, label="После винта")
401
402 for face in side_faces
403     fx = [trans_x[i] for i in face]
404     fy = [trans_y[i] for i in face]

```

```

405         fz = [trans_z[i] for i in face]
406         plot!(p, fx, fy, fz, color=:blue, linewidth=2, alpha=0.7,
               ↪ label="")
407     end
408     scatter!(p, trans_x, trans_y, trans_z, color=:blue,
               ↪ markersize=6, label="")
409
410     refl_x = [pt[1] for pt in reflected]
411     refl_y = [pt[2] for pt in reflected]
412     refl_z = [pt[3] for pt in reflected]
413
414     refl_base_x = [refl_x[1], refl_x[2], refl_x[3], refl_x[4],
                    ↪ refl_x[1]]
415     refl_base_y = [refl_y[1], refl_y[2], refl_y[3], refl_y[4],
                    ↪ refl_y[1]]
416     refl_base_z = [refl_z[1], refl_z[2], refl_z[3], refl_z[4],
                    ↪ refl_z[1]]
417
418     plot!(p, refl_base_x, refl_base_y, refl_base_z,
            ↪ color=:violet, linewidth=2, label="Отражённая")
419
420     for face in side_faces
421         fx = [refl_x[i] for i in face]
422         fy = [refl_y[i] for i in face]
423         fz = [refl_z[i] for i in face]
424         plot!(p, fx, fy, fz, color=:violet, linewidth=2,
               ↪ alpha=0.7, label="")
425     end
426     scatter!(p, refl_x, refl_y, refl_z, color=:violet,
               ↪ markersize=6, label="")
427
428     labels = ["A", "B", "C", "D", "S"]
429     offset = 0.12
430     for i in 1:length(original)
431         annotate!(p, orig_x[i]+offset, orig_y[i]+offset,
                  ↪ orig_z[i]+offset, text("$(labels[i])", :red, 12,
                  ↪ :right, strokecolor=:white, strokewidth=2))
432     annotate!(p, trans_x[i]-offset, trans_y[i]-offset,
               ↪ trans_z[i]-offset, text("$(labels[i])", :blue, 12, :left,
               ↪ strokecolor=:white, strokewidth=2))
433         annotate!(p, refl_x[i], refl_y[i]+offset,
                  ↪ refl_z[i]+offset, text("$(labels[i])", :violet, 12,
                  ↪ :center, strokecolor=:white, strokewidth=2))
434     end
435
436     return p
437 end

```

```

438
439 begin
440     @bind nx Slider(-1:0.01:1, default=0.0, show_value=true)
441     @bind ny Slider(-1:0.01:1, default=0.0, show_value=true)
442     @bind nz Slider(-1:0.01:1, default=1.0, show_value=true)
443     @bind d_plane Slider(-2:0.1:2, default=0.5, show_value=true)
444
445     md"""
446     ### Настройка плоскости отражения
447
448     **Нормальный вектор:**
449     - nx: $(@bind nx Slider(-1:0.01:1, default=0.0,
450       ↪ show_value=true))
451     - ny: $(@bind ny Slider(-1:0.01:1, default=0.0,
452       ↪ show_value=true))
453     - nz: $(@bind nz Slider(-1:0.01:1, default=1.0,
454       ↪ show_value=true))
455
456     **Расстояние от начала координат:**
457     - d: $(@bind d_plane Slider(-2:0.1:2, default=0.5,
458       ↪ show_value=true))
459
460     """
461 end
462
463 begin
464     plot_result = plot_pyramids(original_points,
465       ↪ transformed_points, reflected_points, normal_vec, d_plane)
466
467     savefig(plot_result, "пирамида_преобразование.pdf")
468
469     println("Исходные точки пирамиды:")
470     labels = ["A", "B", "C", "D", "S"]
471     for (i, p) in enumerate(original_points)
472         println("$labels[i]: $p")
473     end
474
475     println("\nПосле винтового движения (вращение 180° вокруг Oz +
476       ↪ трансляция на 1 вдоль Oz):")
477     for (i, p) in enumerate(transformed_points)
478         println("$labels[i]': $p")
479     end
480
481     println("\nТекущая плоскость отражения: $(round.(normal_vec,
482       ↪ digits=2)) · r = $(round(d_plane, digits=2))")
483
484     plot_result

```

478 end

479

5.4. Приложение №4

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import math
4
5 from math import cos, sin
6
7 from dataclasses import dataclass
8
9
10 @dataclass
11 class Dual:
12     real : float
13     dual : float = 0
14
15     def __add__(self, y):
16         if not isinstance(y, Dual):
17             y = Dual(y, 0)
18         return Dual(self.real + y.real, self.dual + y.dual)
19
20     def __sub__(self, y):
21         if not isinstance(y, Dual):
22             y = Dual(y, 0)
23         return Dual(self.real - y.real, self.dual - y.dual)
24
25     def __mul__(self, y):
26         if not isinstance(y, Dual):
27             y = Dual(y, 0)
28         return Dual(self.real * y.real, (self.real * y.dual) +
29             ↪ (self.dual * y.real))
30
31     def __truediv__(self, y):
32         if not isinstance(y, Dual):
33             y = Dual(y, 0)
34         y_real_inv = 1 / y.real
35         real_div = self.real * y_real_inv
36         return Dual(real_div, (self.dual - real_div*y.dual) *
37             ↪ y_real_inv)
38
39     def __repr__(self):
40         if self.dual >= 0:
```

```

40         return f'{self.real} + {self.dual}ε'
41     else:
42         return f'{self.real} + {-self.dual}ε'
43
44     @staticmethod
45     def cos(A):
46         "Косинус от дуального числа"
47         return Dual(cos(A.real), -sin(A.real) * A.dual)
48
49     @staticmethod
50     def sin(A):
51         "Синус от дуального числа"
52         return Dual(sin(A.real), cos(A.real) * A.dual)
53
54     # Статический метод вызывается не из объекта, а из класса
55     print(Dual.cos(Dual(math.pi/2, 1)))
56     print(Dual.sin(Dual(math.pi/2, 1)))
57
58     # Точка O (начало координат)
59     O = np.array([0, 0, 1])
60     # Направляющий вектор прямой
61     v = np.array([2, 1, 0])
62     # Вычисление момента для прямой, проходящей через точку P с
63     ↪ направляющим вектором v
64     p = O
65     m = np.cross(p, v)
66     # Определяем винт, путем задания его дуальных координат.
67     # Действительную часть берем из вектора v, а дуальную часть из
68     ↪ момента m
69     L = np.array([Dual(v[i], m[i]) for i in range(3)], dtype=Dual)
70     # Теперь L – винт, задающий прямую
71
72     # Направляющий вектор оси Oz
73     OZ = np.array([0, 0, 1])
74     # Зададим теперь ось вращения в виде винта
75     A = np.array([Dual(v, m) for (v, m) in zip(OZ, np.cross(O, OZ))],
76     ↪ dtype=Dual)
77
78     print("Вектор v:", v)
79     print("Вектор p:", p)
80     print("Вектор m (момент):", m)
81     print(f"Винт L = v + ε * m = {L}")
82     print(f"Винт A = v + ε * m = {A}")
83
84     def Rodrig(A, θ, L):
85         return A * (Dual(1, 0) - Dual.cos(θ)) * np.dot(A, L) + L *
86         ↪ Dual.cos(θ) + np.cross(A, L) * Dual.sin(θ)

```

```

83
84 result = Rodrig(A,  $\Theta$ , L)
85 print(f"Результат функции Rodrig:\n{result}")
86
87  $\Theta$  = Dual(math.pi / 2, 1) # дуальный угол
88
89 # Вычисление нового вектора
90 L1 = Rodrig(A,  $\Theta$ , L)
91
92 print(f"L1' = {L1}")
93
94 real_parts = np.array([component.real for component in L1])
95 dual_parts = np.array([component.dual for component in L1])
96
97 print("Действительные части:", real_parts)
98 print("Дуальные части:", dual_parts)
99
100
101 #Соответственно  $v'$  обозначим, как  $V$ 
102 V = real_parts
103 V
104
105 #  $m$  обозначим как  $M$ 
106 M = dual_parts
107 M
108
109 # Вычисление нормы вектора  $v$ 
110 norm = np.linalg.norm(v)
111
112 # Нормализация вектора
113 normalized_v = v / norm
114
115 print("Нормированный вектор:", normalized_v)
116
117
118 # Соответствует  $[1/\sqrt{2} \ 1/\sqrt{2} \ 0]$  так, как  $1/\sqrt{2} = 0.707\dots$ 
119
120
121 # Вычисление нормы вектора  $M$ 
122 norm = np.linalg.norm(M)
123
124 # Нормализация вектора
125 normalized_M = M / norm
126
127 # Вывод результатов
128 print("Нормированный вектор:", normalized_M)
129

```

```

130
131 # Соответствует  $[-1 \ 0 \ 0]$  так, как  $7.85046229e-17$  является очень
    ↪ маленьким числом, близким к нулю.
132
133
134 # Вычисление нормы вектора V
135 norm = np.linalg.norm(V)
136
137 # Нормализация вектора
138 normalized_V = V / norm
139
140 # Вывод результатов
141 print("Нормированный вектор:", normalized_V)
142
143 # Соответствует  $[0 \ 1 \ 0]$  так, как  $7.85046229e-17$  является очень
    ↪ маленьким числом, близким к нулю.
144
145
146 P = np.cross(V, M) / np.linalg.norm(V)**2
147 P
148
149
150 # Прямая L1 будет проходить через точку  $[0 \ 0 \ -1]$ 
151
152
153 fig = plt.figure(num=1, figsize=(8, 8), dpi=200)
154 ax = fig.add_subplot(111, projection='3d')
155
156 ax.set_xlim([-5, 5])
157 ax.set_ylim([-5, 5])
158 ax.set_zlim([-5, 5])
159
160 axis_length = 5
161
162 ax.plot([0, axis_length], [0, 0], [0, 0], color='red',
    ↪ linewidth=1.5, linestyle='-', zorder=3) # X+
163 ax.plot([0, 0], [0, axis_length], [0, 0], color='green',
    ↪ linewidth=1.5, linestyle='-', zorder=3) # Y+
164 ax.plot([0, 0], [0, 0], [0, axis_length], color='blue',
    ↪ linewidth=1.5, linestyle='-', zorder=3) # Z+
165
166 ax.plot([0, -axis_length], [0, 0], [0, 0], color='red',
    ↪ linewidth=1.5, linestyle='--', zorder=3) # X-
167 ax.plot([0, 0], [0, -axis_length], [0, 0], color='green',
    ↪ linewidth=1.5, linestyle='--', zorder=3) # Y-
168 ax.plot([0, 0], [0, 0], [0, -axis_length], color='blue',
    ↪ linewidth=1.5, linestyle='--', zorder=3) # Z-

```

```

169
170 ax.scatter(*O, s=30, c='k', label='Точка O (0, 0, 0)')
171 ax.scatter(*(v + O), s=30, c='purple', label=f'Вектор прямой v')
172 ax.scatter(*(P + O), s=30, c='red', label=f'Точка P')
173
174 t = np.linspace(-5, 5, 100)
175 x = O[0] + v[0] * t
176 y = O[1] + v[1] * t
177 z = O[2] + v[2] * t
178
179 ax.plot(x, y, z, label='L', linewidth=2)
180
181 t2 = np.linspace(-5, 5, 100)
182 x2 = P[0] + normalized_V[0] * t2
183 y2 = P[1] + normalized_V[1] * t2
184 z2 = P[2] + normalized_V[2] * t2
185 ax.plot(x2, y2, z2, label='L1', linewidth=2)
186
187
188 ax.set_xlim([-5, 5])
189 ax.set_ylim([-5, 5])
190 ax.set_zlim([-5, 5])
191
192 axis_length = 5
193
194 ax.set_xlabel('X')
195 ax.set_ylabel('Y')
196 ax.set_zlabel('Z')
197
198 ax.legend(fontsize=10)
199
200 ax.view_init(elev=20, azimuth=30)
201
202 plt.tight_layout()
203
204 plt.savefig('graph.pdf', format='pdf')
205
206 plt.show()
207
208 from ipywidgets import FloatSlider, VBox, interactive
209 from IPython.display import display
210
211 def update_dual(real_part, dual_part):
212      $\Theta$  = Dual(real_part, dual_part) # дуальный угол
213     L1 = Rodrig(A,  $\Theta$ , L)
214
215     real_parts = np.array([component.real for component in L1])

```

```

216 dual_parts = np.array([component.dual for component in L1])
217
218 plt.clf()
219
220 fig = plt.figure(num=1, figsize=(8, 8), dpi=200)
221 ax = fig.add_subplot(111, projection='3d')
222
223 ax.set_xlim([-5, 5])
224 ax.set_ylim([-5, 5])
225 ax.set_zlim([-5, 5])
226
227 axis_length = 5
228
229 ax.plot([0, axis_length], [0, 0], [0, 0], color='red',
    ↪ linewidth=1.5, linestyle='-', zorder=3) # X+
230 ax.plot([0, 0], [0, axis_length], [0, 0], color='green',
    ↪ linewidth=1.5, linestyle='-', zorder=3) # Y+
231 ax.plot([0, 0], [0, 0], [0, axis_length], color='blue',
    ↪ linewidth=1.5, linestyle='-', zorder=3) # Z+
232
233 ax.plot([0, -axis_length], [0, 0], [0, 0], color='red',
    ↪ linewidth=1.5, linestyle='--', zorder=3) # X-
234 ax.plot([0, 0], [0, -axis_length], [0, 0], color='green',
    ↪ linewidth=1.5, linestyle='--', zorder=3) # Y-
235 ax.plot([0, 0], [0, 0], [0, -axis_length], color='blue',
    ↪ linewidth=1.5, linestyle='--', zorder=3) # Z-
236
237 ax.scatter(*0, s=30, c='k', label='Точка 0 (0, 0, 0)')
238 ax.scatter(*(v + 0), s=30, c='purple', label='Вектор линии v')
239
240 P = np.cross(real_parts, dual_parts) /
    ↪ np.linalg.norm(real_parts)**2
241 ax.scatter(*(P + 0), s=30, c='red', label='Точка P')
242
243 t = np.linspace(-5, 5, 100)
244 x = 0[0] + v[0] * t
245 y = 0[1] + v[1] * t
246 z = 0[2] + v[2] * t
247
248 ax.plot(x, y, z, label='Прямая L', linewidth=2)
249
250 t2 = np.linspace(-5, 5, 100)
251 x2 = P[0] + real_parts[0] * t2
252 y2 = P[1] + real_parts[1] * t2
253 z2 = P[2] + real_parts[2] * t2
254
255 ax.plot(x2, y2, z2, label='Прямая L1', linewidth=2)

```

```

256
257     ax.set_xlabel('X')
258     ax.set_ylabel('Y')
259     ax.set_zlabel('Z')
260     ax.legend(fontsize=10)
261
262     ax.view_init(elev=20, azim=30)
263     plt.tight_layout()
264     plt.show()
265
266     real_slider = FloatSlider(value=0.real, min=0, max=math.pi / 4,
    ↪     step=0.01, description='Real part:')
267     dual_slider = FloatSlider(value=0.dual, min=0, max=1, step=0.01,
    ↪     description='Dual part:')
268
269     interactive_plot = interactive(update_dual, real_part=real_slider,
    ↪     dual_part=dual_slider)
270     output = interactive_plot.children[-1]
271     output.layout.height = '400px'
272
273     display(real_slider, dual_slider, output)

```

6. Заключение

В ходе выполнения работы получены следующие основные результаты.

Изучены основные концепции кватернионов Гамильтона и бикватернионов, которые применимы к описанию пространственных преобразований. Рассмотрены алгебра кватернионов в нотации однородных координат, операции сопряжения, скалярное и векторное произведения, а также представление единичного кватерниона в тригонометрической форме. На этой основе изложены формулы вращения точки и вектора вокруг оси, проходящей через начало координат, с помощью сэндвич-формулы.

Рассмотрено определение дуальных бикватернионов тремя способами: через таблицу умножения базисных элементов, через процедуру удвоения Кейли–Диксона и через дуальное представление кватернионных формул. Введены операции над бикватернионами, включая три типа сопряжения, скалярное и винтовое произведения, модуль и параметр бикватерниона. Представлены и другие объекты в пространстве на которые можно воздействовать винтовым движением: аффинная точки, вектор, прямая и плоскость, которые впоследствии можно представить в виде бикватернионов.

На основе принципа перенесения Котельникова–Штуди построены бикватернионные формулы винтового движения. Получены явные выражения для бикватерниона винтового преобразования, разложенного на вращательную и трансляционную составляющие. Выведены сэндвич-формулы для винтового движения точки, вектора, прямой и плоскости. Показана связь бикватернионного представления преобразований с матрицами проективных преобразований в трёхмерном пространстве. Дополнительно рассмотрены формулы отражения точки, плоскости и прямой относительно произвольной плоскости с помощью бикватернионов.

Основные операции над бикватернионами и функции были реализованы в качестве блока на Julia. Блок содержит полный набор операций над бикватернионами и функций для описания пространственного перемещения объекта вокруг произвольной и фиксированной осей, а также отражения относительно произвольной плоскости. Использование Julia обеспечивает высокую производительность вычислений, что позволяет применять библиотеку в задачах, требующих численного моделирования в реальном времени. Работоспособность программ проверена на тестовых примерах винтового движения точек. В качестве результата вычислений каждая программа выводит наглядный график.

Таким образом, разработанные программы удобны для вычисления и моделирования пространственных преобразований.

Список литературы

1. Dual Numbers and Automatic Differentiation. — URL: https://github.com/rlabbe/dual_numbers/blob/main/dual_numbers.ipynb (дата обр. 15.03.2025).
2. Информационно-телекоммуникационные технологии и математическое моделирование высокотехнологичных систем // Материалы Всероссийской конференции с международным участием. Москва, РУДН, 7–11 апреля 2025 г. — Москва : РУДН, 2025. — ISBN 9785209123835. — ил.
3. *Goldman R.* Dual Quaternions and Their Associated Clifford Algebras. — Boca Raton : CRC Press, 2023. — 289 с. — ISBN 978-1-032-38907-7.
4. *Гордеев В. Н.* Кватернионы и бикватернионы с приложениями в геометрии и механике. — Москва : Сталь, 2016. — 316 с. — ISBN 9786176760993.
5. *Lengyel E.* Mathematics for 3D Game Programming and Computer Graphics. — 3-е изд. — Course Technology, Cengage Learning, 2012. — ISBN 9781435458864.
6. *Goldman R.* Rethinking Quaternions: Theory and Computation. — Morgan & Claypool, 2010. — (Synthesis Lectures on Computer Graphics and Animation ; 13). — ISBN 9781608454204.
7. *Котельников А. П.* Винтовое счисление и некоторые приложения его к геометрии и механике. — Москва : КомКнига, 2006. — 224 с. — ISBN 978-5-484-00657-1.
8. *Челноков Ю. Н.* Кватернионные и бикватернионные модели и методы механики твердого тела и их приложения. Геометрия и кинематика движения. — Москва : ФИЗМАЛИТ, 2006. — 512 с. — ISBN 5922106805.
9. *Kuipers J. B.* Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality. — Princeton, New Jersey : Princeton University Press, 1999. — 371 с. — ISBN 0691058725.
10. *Ziegler R.* Die Geschichte der geometrischen Mechanik im 19. Jahrhundert: eine historisch-systematische Untersuchung von Möbius und Plücker bis zu Klein und Lindemann. — Stuttgart : Franz Steiner Verlag, 1985. — (Boethius ; 13). — ISBN 3515045449.
11. *Бранцеу В. Н., Шмыглевский И. П.* Применение кватернионов в задачах ориентации твердого тела. — Москва : Наука, 1973. — 320 с.
12. *Диментберг Ф. М.* Винтовое исчисление и его приложения в механике. — Москва : Наука, 1965. — 199 с. — ISBN 43 к.
13. *Лурье А. И.* Аналитическая механика. — Москва : Государственное издательство физико-математической литературы, 1961. — 824 с.